

Conception Orientée Objet

- *Principes*
- *Le langage UML 2.0*

Plan

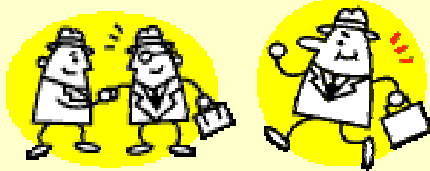
- Introduction
- Rappels de l'Orienté-Objet
- Développement orienté objet
- Présentation d'UML
- Les cas d'utilisation
- Les diagrammes de classes
- Les diagrammes de séquence
- Les diagrammes d'états-transitions
- Autres diagrammes d'UML

Introduction

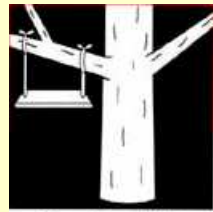
Ingénierie des systèmes, dites-vous ?

Client & informaticien

Dialogue de sourds ??



Ce que voulait
le client



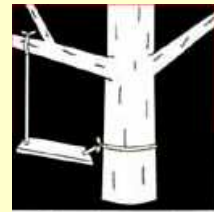
Ce qu'a spécifié
le client



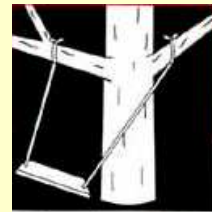
Ce qu'a promis
l'ingénieur commercial



Ce qu'a compris
le chef de projet



Ce qu'a compris
le concepteur



Ce qui a été livré
Version 1



Ce qui fonctionne
actuellement
Version 1 + patch
Rebaptisée Version Béta



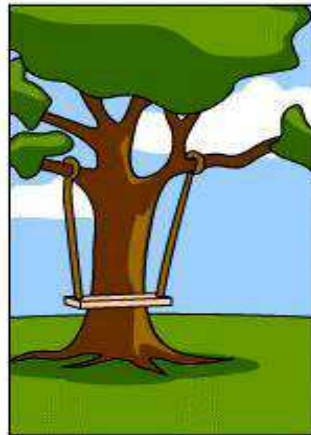
Inspiré de <http://www.bric-a-brac.org/humour/images/informatique>

Client & informaticien

Dialogue de sourds ??



How the customer explained it



How the Project Leader understood it



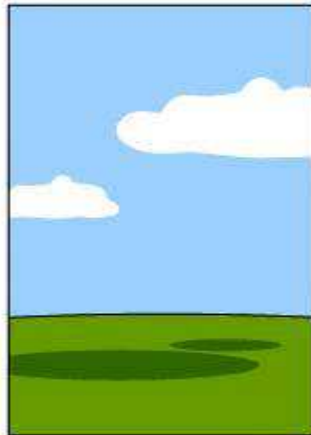
How the Analyst designed it



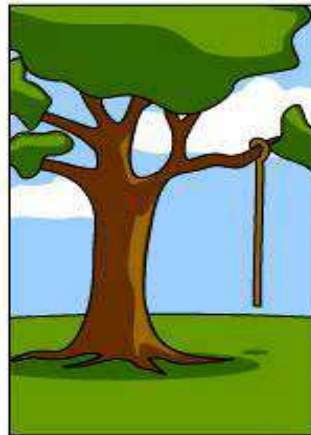
How the Programmer wrote it



How the Business Consultant described it



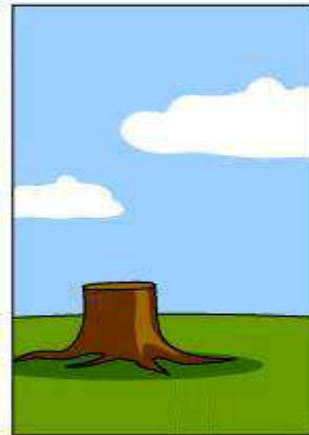
How the project was documented



What operations installed



How the customer was billed



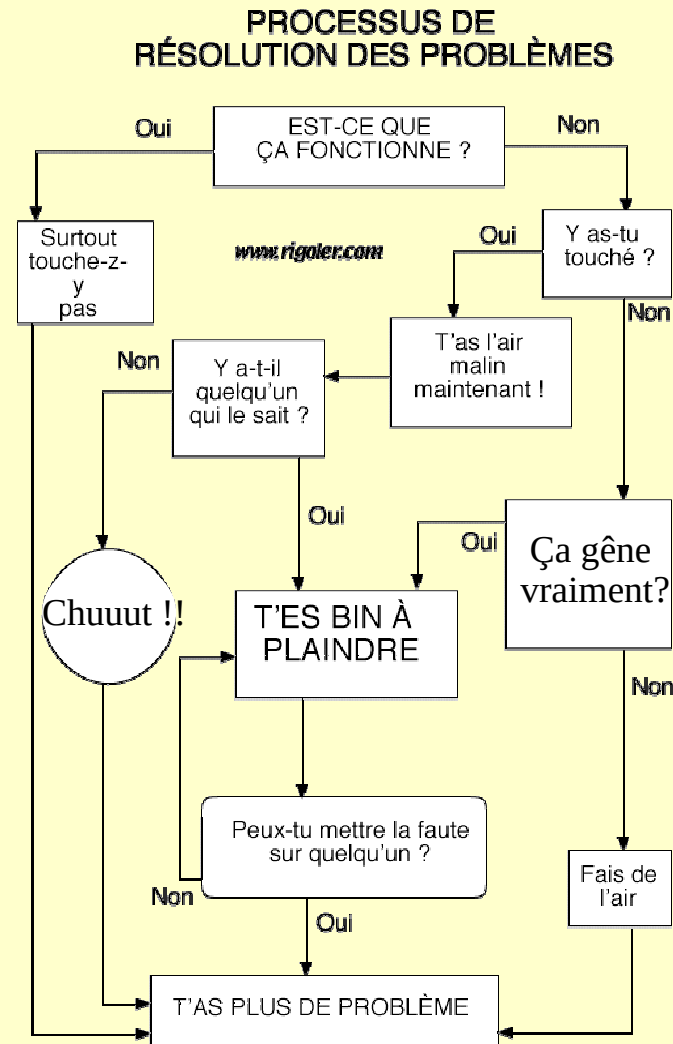
How it was supported



What the customer really needed

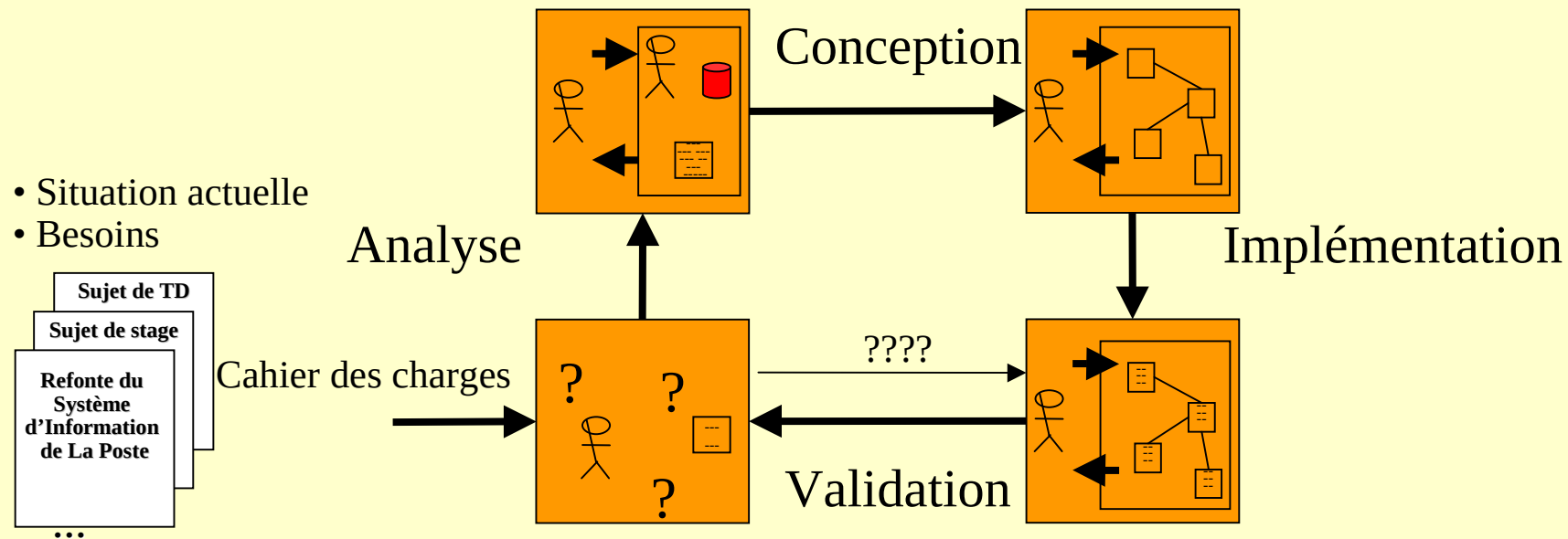
Pardon ? Ingénierie des systèmes?

Analyse d'un système existant :
quelle démarche adopter ??



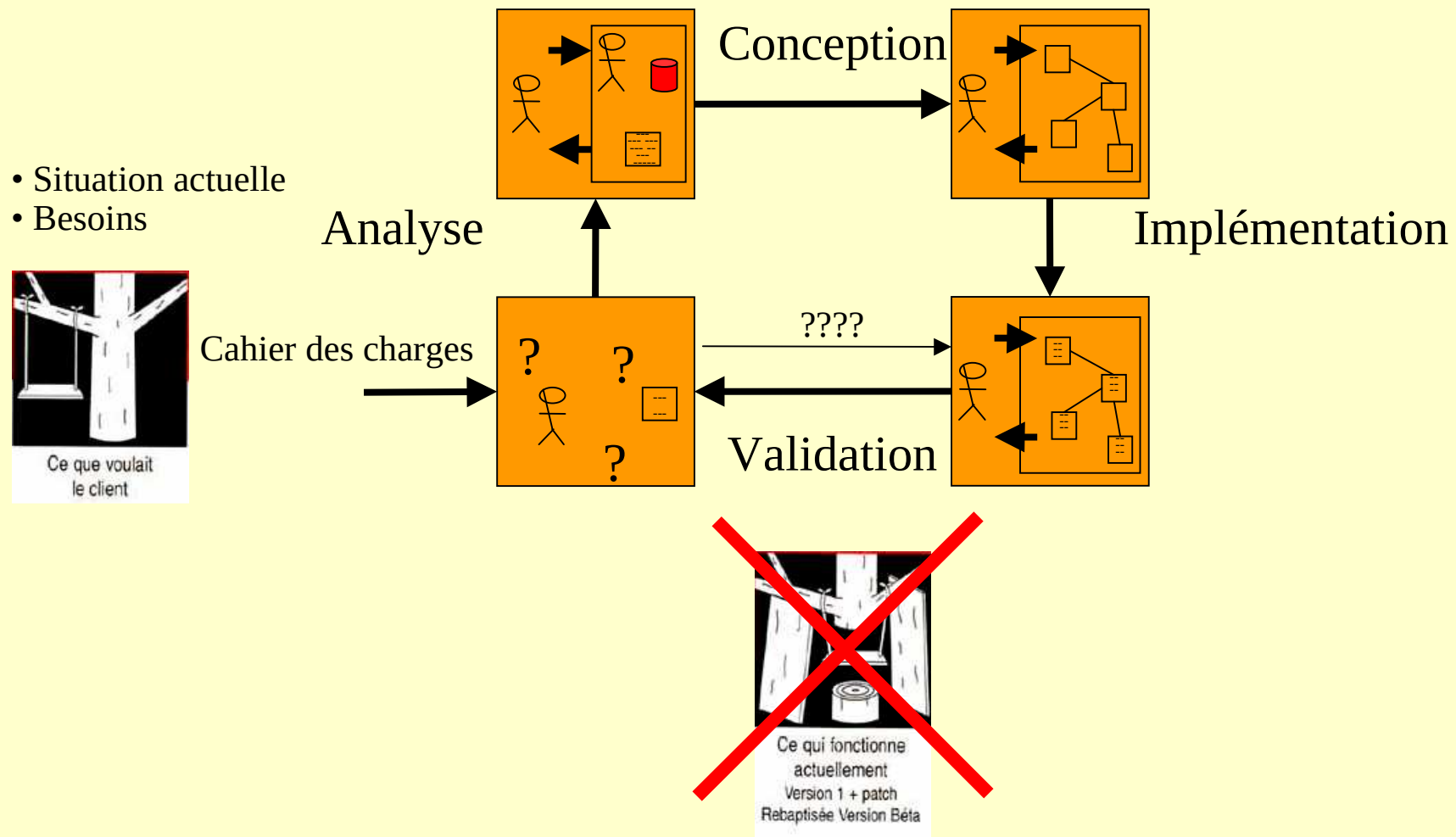
<http://www.bric-a-brac.org/humour/images/informatique>

Ingénierie des systèmes : processus



- **Analyse** : abstraction et extraction des besoins
- **Conception** : concrétisation + solutions organisationnelles et techniques
- **Implémentation** : adaptation, codage, installation (logiciels + matériel)

Ingénierie des systèmes : processus



Objet & Conception

- L'objet : avantages certains
- L'objet : introduit avec la programmation.
- Trop lié aux langages de programmation.
- Analyser et concevoir 'objet' : nécessité
 - d'une modélisation conceptuelle
 - d'un détachement de toute plate-forme de programmation
 - apparition de méthodes de conception objet

Objet & Conception

- Analyser et concevoir 'objet' :
 - méthodes objet
 - Avantages de l'approche objet : classes, objets, ...
 - Insertion d'expressions sémantiques : les associations entre classes.

Conception orientée objet

Principes

Conception objet : principes

- Méthodologie importante : penser Objet !
- Notion de *réification* : comment faire de quelque chose un objet ?
- L'*autonomie* et la *localité* : comment penser dans les termes propres de l'objet ?
- La *composition* et l'*affinage* : comme modes de structuration d'objets.
- La *généricité* : comment développer des applications de manière abstraite ?

Conception objet :

- Méthodes classiques de conception :
 - Approche totalement descendante.
- Méthode objet de conception :
 - Approche plutôt :
 - incrémentale.
 - itérative.
 - Spécifications par affinages successifs.

Conception objet :

- Une démarche itérative et incrémentale ?
 - modéliser : comprendre et représenter un système.
 - en plusieurs fois en affinant son analyse par étapes.
 - pour mieux maîtriser la part d'inconnu et d'incertitudes caractérisant les systèmes complexes.

Conception : étapes générales

- Identifier les entités du domaine.
- Structurer le domaine : en analysant les propriétés de ces entités et leurs relations.
- Identifier les opérations que savent effectuer ces entités
- Décrire précisément ces opérations en les reliant à des messages.
- Décrire le lancement du programme.

Quelques conseils

- Savoir revenir aux étapes précédentes.
- Prototyper rapidement avant de passer à une spécification complète.
- Penser « abstrait ».
- Penser « local ».
- Les opérations peuvent-elles être des objets ?

Les erreurs communes

- Faire des tests répétés.
- Confondre héritage et composition.
- Manipuler des objets « au loin ».
- Demander la classe de l'objet avant d'agir.
- Penser en termes de fonctions et ne pas penser en termes d'objets.

Le langage UML 2.0



Unified Modeling Language

Modèle ?

- *Abstraction d'une entité du monde réel pour:*
 - *la comprendre*
 - *avant de la construire*
- Tester avant de construire
- Communiquer avec les clients
- Visualiser
- Réduire la complexité

Modèle orienté-objet?

- C'est un ... modèle !
- Permet en plus :
 - Abstraction : l'essentiel sans le détail
 - Encapsulation : 'masquage' de l'information interne
 - Regroupement des données et du comportement
 - Partage : héritage – spécialisation

UML : présentation

- « *The Unified Modeling Language (UML) is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML offers a standard way to write a system's blueprints, including conceptual things such as business processes and system functions as well as concrete things such as programming language statements, database schemas, and reusable software components.* » [OMG]

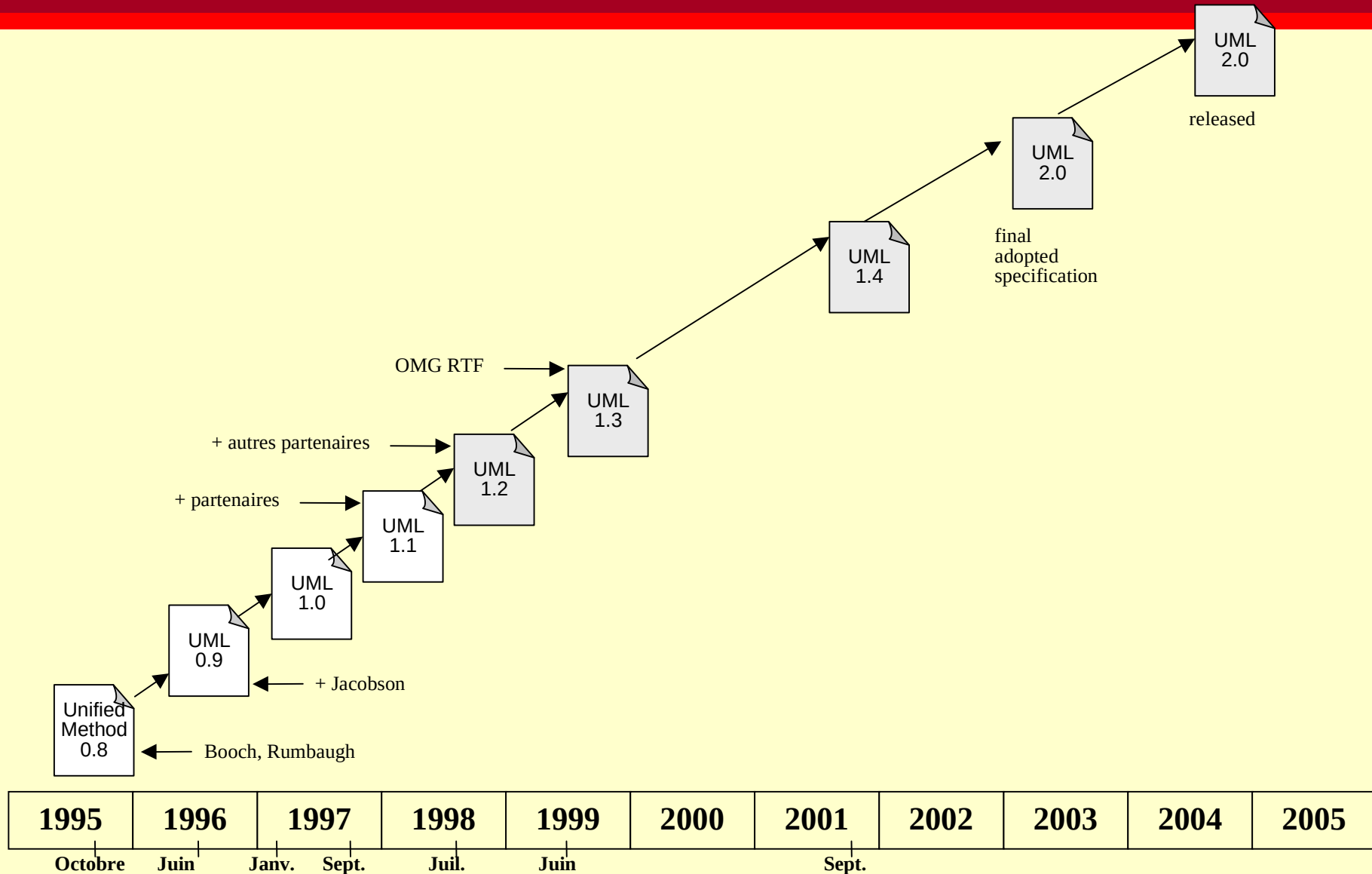
UML : présentation

- UML :
 - langage de modélisation
 - notation graphique
 - création de modèles orientés objet
 - pour :
 - l'analyse
 - la modélisation de logiciels orientés objet

UML : présentation

- UML est un langage de modélisation visuel ayant un ensemble de notations
- UML n'est pas une méthode d'analyse ou de conception
- Une méthode de conception propose :
 - Un langage de modélisation (graphique, formel...)
 - Une démarche de modélisation

Historique



Unified Modelling Language

- S'utilise :
 - avec toute méthode de développement.
 - dans tous domaines d'application.
- Langage « à la mode » car c'est un standard
- Les utilisateurs guident la définition des modèles et le cycle de développement

UML : utilité

- Communication : l'important sans le détail
 - Manipulation graphique de concepts au sein de diagrammes
 - Manipulation graphique des connexions entre diagrammes
- Communication avec :
 - Les clients
 - Les experts
 - Soi-même !!!
- Apprentissage de l'approche objet :
 - Diagramme de classes : hiérarchie de classes
 - Diagrammes d'interactions : entre objets

TOUT COMPRENDRE ET
REPRESENTER EN UNE SEULE
FOIS ?????!!!!

« Rédiger » un modèle avec UML

- UML définit un système avec des diagrammes
- Un diagramme UML :
 - est une représentation graphique,
 - qui s'intéresse à un aspect précis du modèle.
 - C'est une perspective du modèle et non "le modèle".
- Chaque type de diagramme UML (13 diagrammes !) :
 - possède une structure.
 - met en jeu des concepts.
 - les types des éléments de modélisation sont prédéfinis.
- Un type de diagramme UML :
 - sémantique précise.
 - même vue d'un système.

Diagrammes (modèles) UML

- **Diagramme de cas d'utilisation**
(Use Case Diagram)
- **Diagramme de classes**
(Class Diagram)
- **Diagramme de séquences**
(Sequence Diagram)
- **Diagramme d'états-transitions**
(State Diagram)
- **Diagramme de composant & de déploiement**
(Component Diagram)
(Deployment Diagram)
- ...

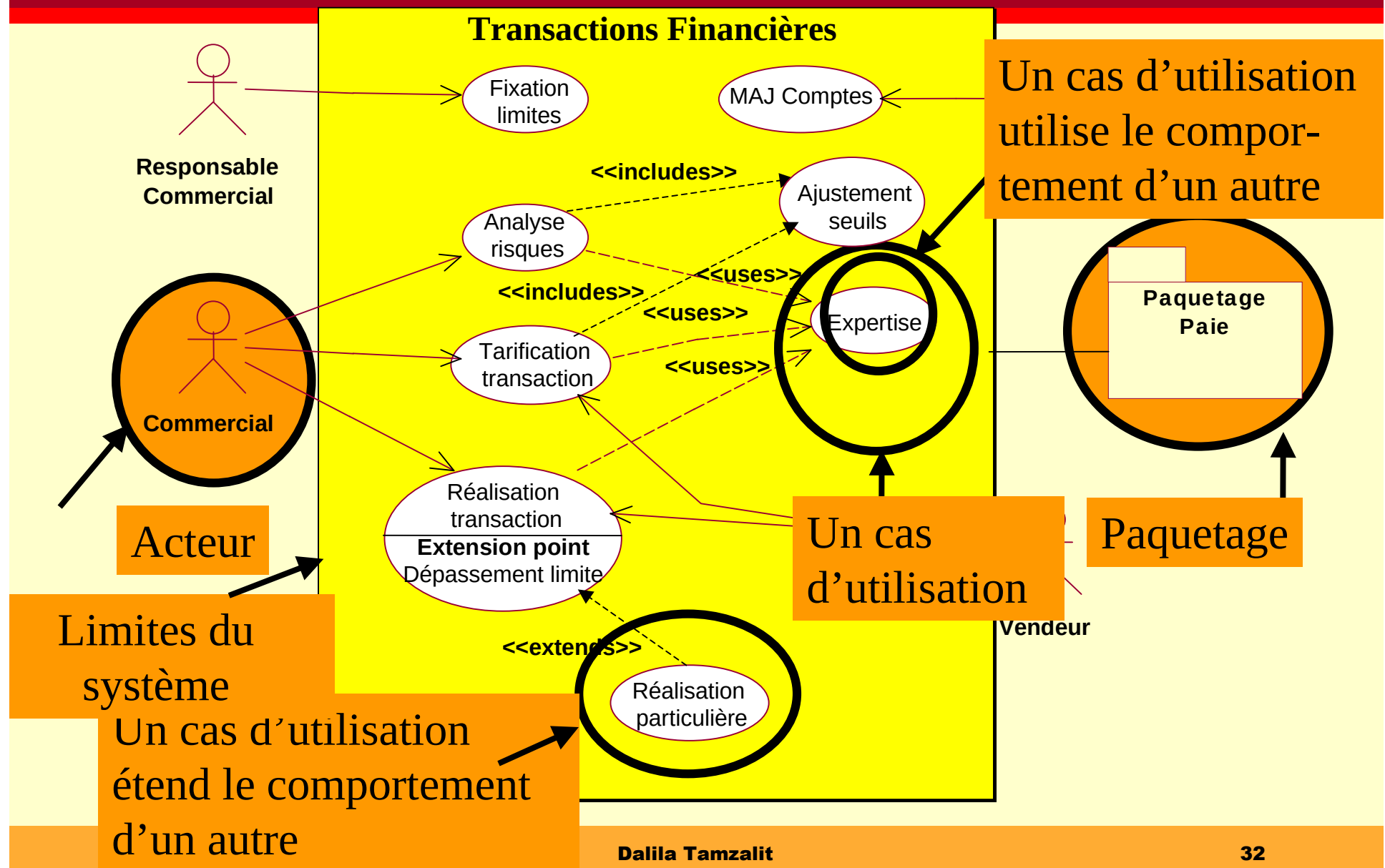
Diagramme de cas d'utilisation

Use Cases Diagram

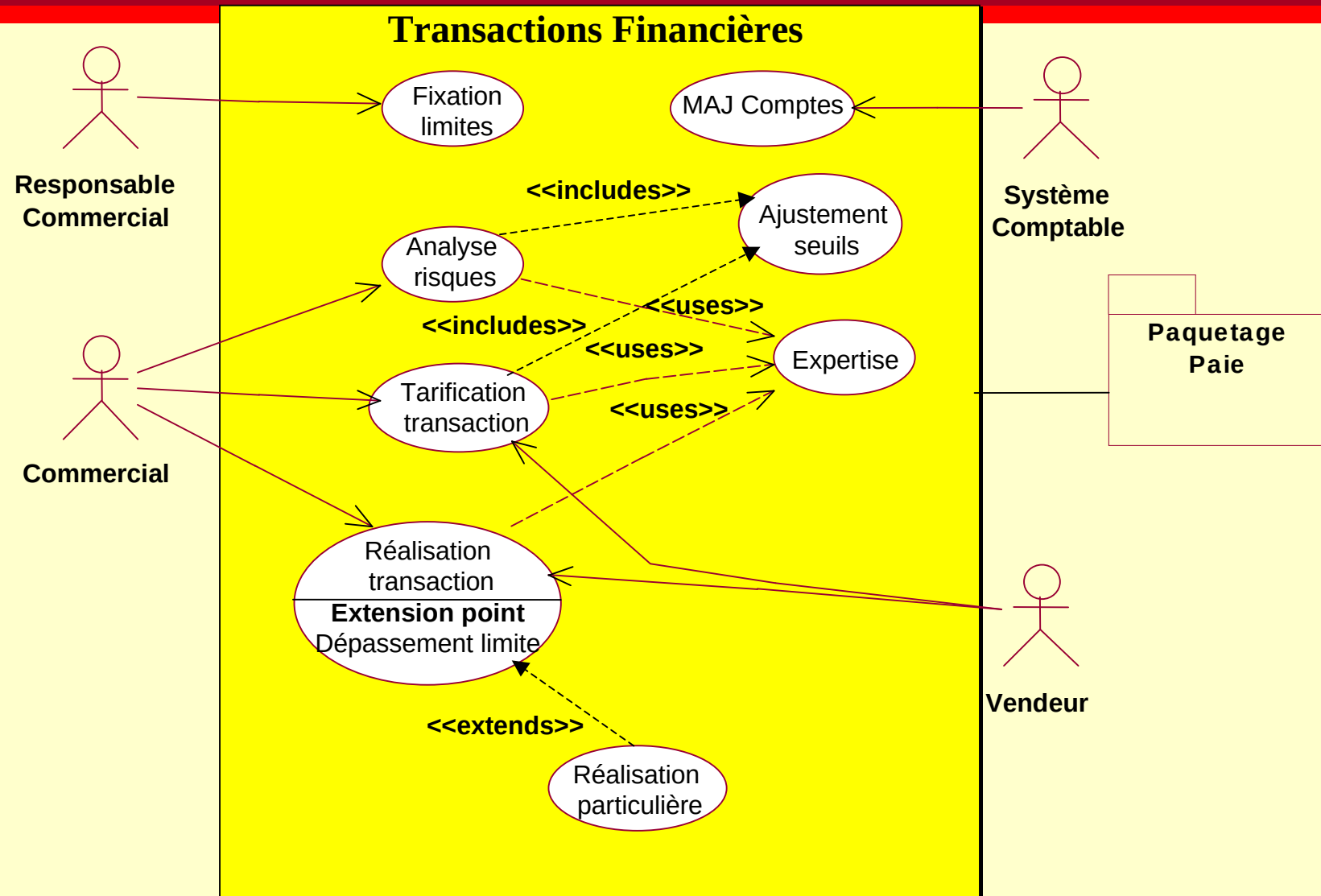
Présentation

- Délimite les frontières du système par rapport à son environnement et aux attentes
- Décrit le comportement d'un **système** par rapport à son environnement et à ses objectifs
- Capture les interactions :
 - entre le système et son environnement (généralement, les utilisateurs)
 - à un niveau d'abstraction élevé
 - En terme de principales fonctionnalités du système
- Comprendre les besoins des utilisateurs

Cas d'utilisation : exemple

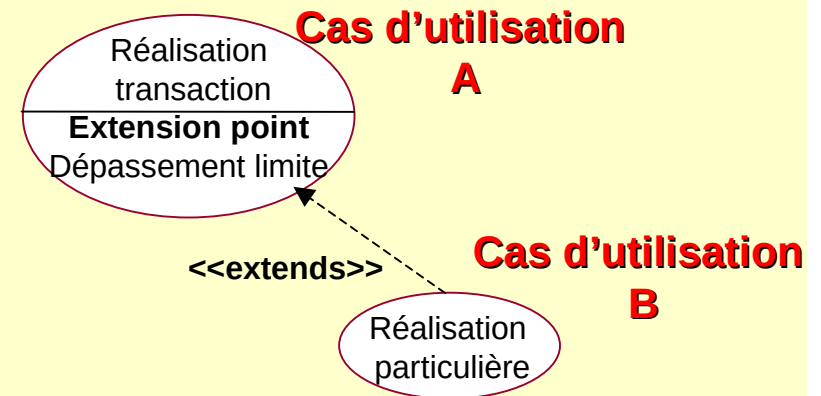


Cas d'utilisation : exemple



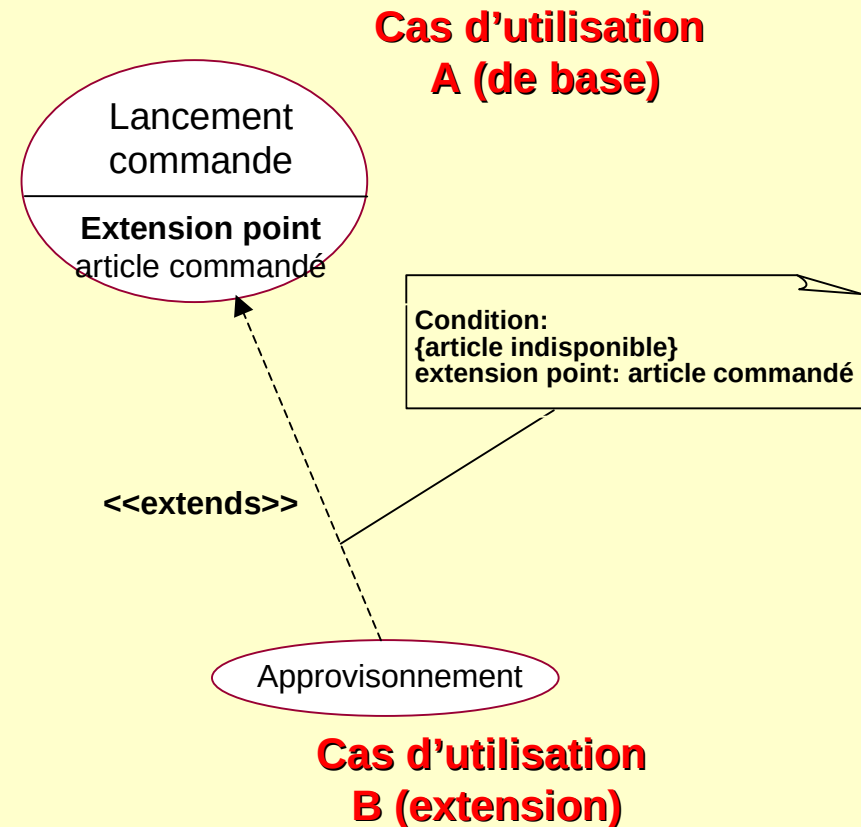
Cas d'utilisation

- A: cas d'utilisation de base
- B: cas d'utilisation spécifique qui étend et complète le cas d'utilisation A
- A peut être exécuté seul ou avec ses extensions
- Les points d'exécution indiquent les emplacements auxquels le cas d'utilisation A est complété



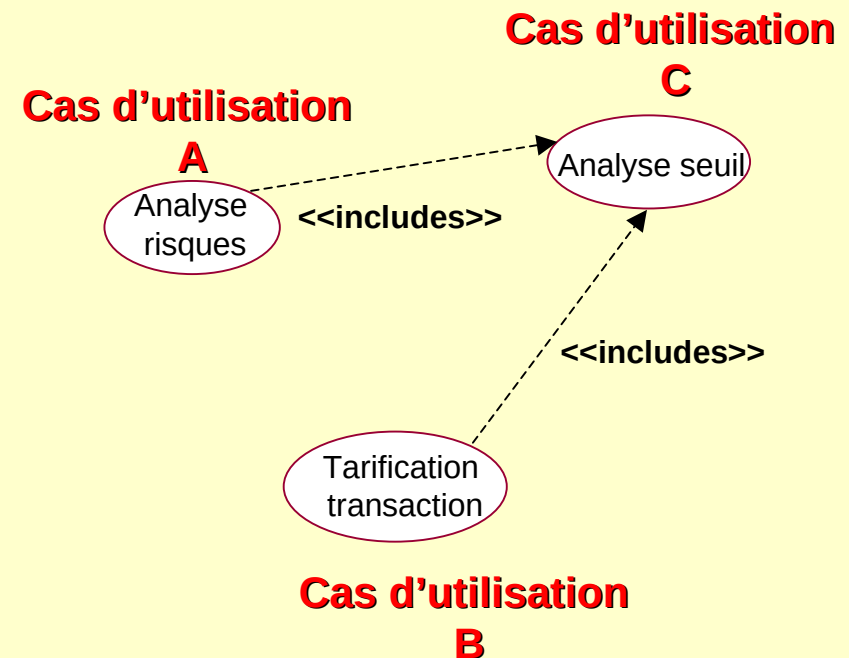
Cas d'utilisation

- Pour ajouter une extension, il faut qu'une condition soit remplie.
- Une condition peut être spécifiée sous forme de note ou de commentaire attaché à la relation `<<extend>>`

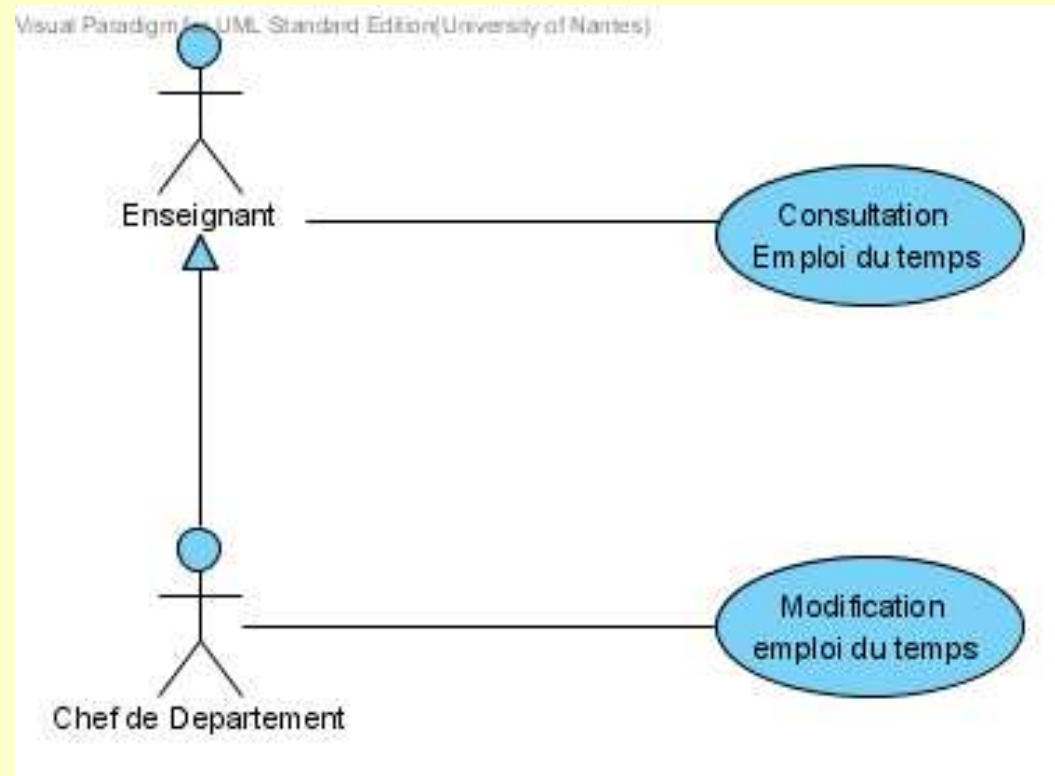


Cas d'utilisation

- La relation `<<includes>>` permet à la fonctionnalité commune des cas d'utilisation A et B d'être décrite par un cas d'utilisation C.
- C est toujours nécessaire pour que A et B puisse s'exécuter.



Lien entre acteurs



Présentation

- Un cas d'utilisation peut avoir plusieurs **scénarios** possibles
- Un cas d'utilisation, donc une fonctionnalité, est déclenché par :
 - Un **acteur**
 - Un autre cas d'utilisation

Cas d'utilisation

- **Système** : application étudiée.
- **Scénario** : suite de séquences décrivant une interaction entre deux entités :
 - Description textuelle : langage naturelle
 - Description interactive : voir diagramme d'interactions
- **Cas d'utilisation** :
 - Un **cas d'utilisation** représente une **fonctionnalité** du système
 - ensemble de scénarios reliés à un objectif commun

Cas d'utilisation

- **Acteur** : entité physique, morale, système externe ... toute entité pouvant déclencher un cas d'utilisation.
- **Diagramme de cas d'utilisation** : ensemble des entités mises en interactions et en relations.

Cas d'utilisation : utilité

- Représentation externe du fonctionnement du système
- Outil essentiel d'un projet itératif pour :
 - Le recueil des besoins
 - La planification
 - Le contrôle
- Un cas d'utilisation ne se restreint *jamais* à un scénario. Il en englobe plusieurs et peut présenter plusieurs variantes pour un scénario.
- Si plusieurs sous-systèmes : un diagramme par sous-système et défini dans un paquetage

Cas d'utilisations: interaction

- Interaction : déclenchement d'un cas d'utilisation
 - Entre un acteur et le système
 - Entre un acteur et un cas d'utilisation
 - Entre un cas d'utilisation et un autre

Cas d'utilisations: relation

- Relation :
 - Extension : un cas d'utilisation peut en étendre un autre. Il en hérite. C'est un sous-cas d'utilisation
 - Utilise : un cas d'utilisation peut en utiliser un ou plusieurs autres.
 - Un cas d'utilisation peut en contenir d'autres

Cas d'utilisation : de plus...

- Possibilité d'avoir des cardinalités sur les relations
- Possibilité d'avoir un nom pour une relation
- Un cas d'utilisation est concrètement déclenché par l'apparition d'un événement
- Un cas d'utilisation peut rendre en sortie un ou plusieurs événements
- L'état du système peut être consulté et/ou modifié par les entités (acteur, cas d'utilisation) autorisé.

Diagramme de classes

Class Diagram

Introduction

- Le diagramme de classes décrit la vue statique du système
- En termes des
 - types d'objets du système :
 - Leurs attributs
 - Leurs méthodes
 - différents types de relations statiques entre eux.

Diagrammes de classes

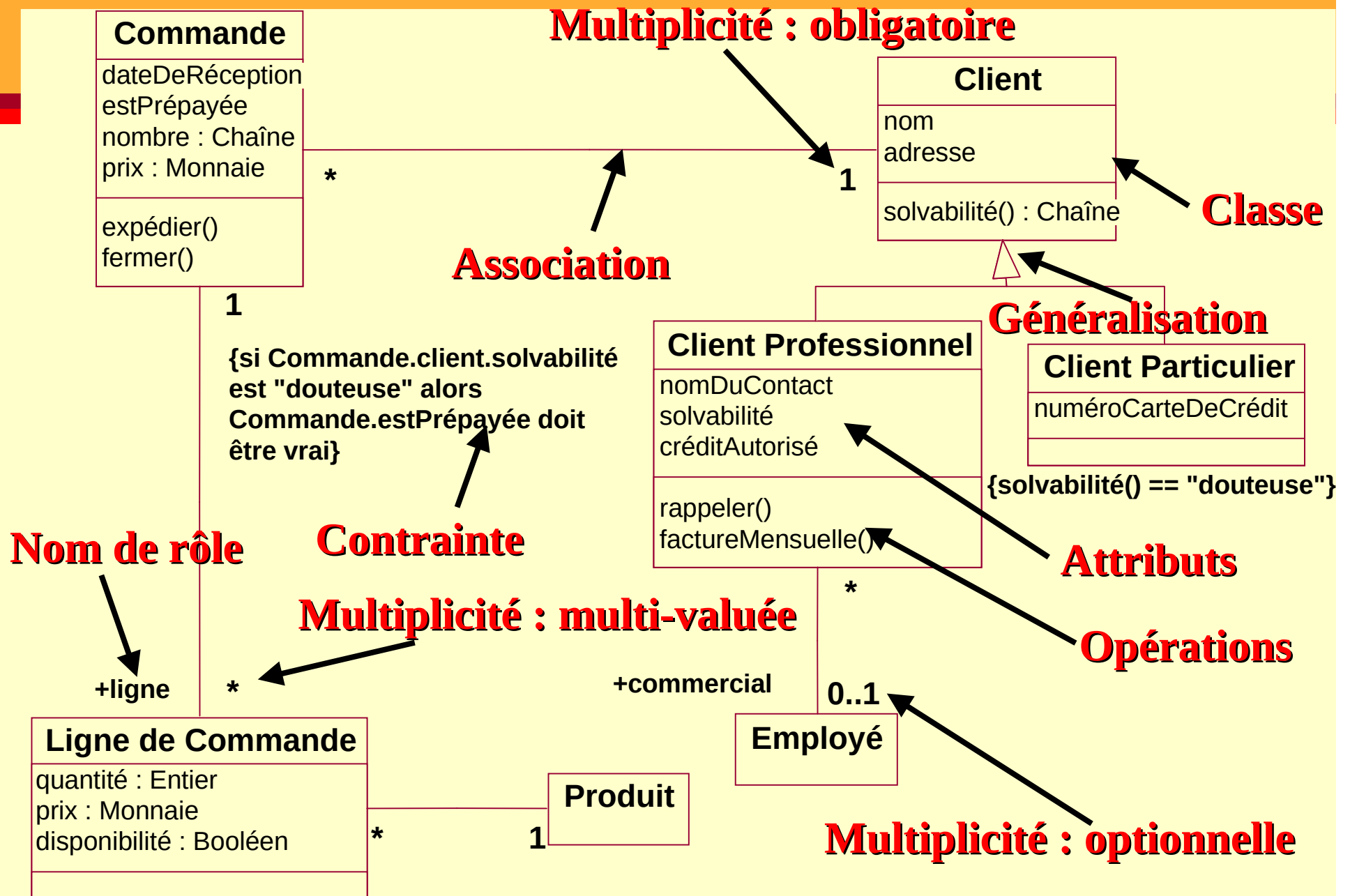
- Concepts majeurs :
 - Classes
 - Associations
 - Généralisation et héritage
 - Agrégation / Composition
 - Navigation
 - Contraintes
 - Attributs – Opérations
- Concepts avancés :
 - Stéréotype
 - Qualification
 - Attributs et opérations de classes
 - Dépendances entre classes
 - Classes abstraites
 - Interfaces
 - Diagramme d'objets

Concepts majeurs: classe

- Classe :
 - Représente des ensembles d'objets caractérisés par des attributs et des opérations similaires.
 - Un état est décrit par des attributs et des associations
 - Une opération pour représenter un comportement
 - Une méthode est l'implémentation d'une opération

Concepts majeurs: association

- Association entre classes :
 - Représente les ensembles de liens similaires entre les objets de ces classes.
- Une classe définit un ensemble d'objets.
- Une association définit un ensemble de liens.





Les concepts : associations

- Association : représente les liens unissant les instances des classes.
 - Une commande doit provenir d'un seul client ; un client peut passer plusieurs commandes.
- Nom d'une association : choix entre
 - Nommer toutes les associations.
 - Nommer celles qui gagneraient à être ainsi plus explicites.

Association : UML vs Java

```
public class Commande
{
```

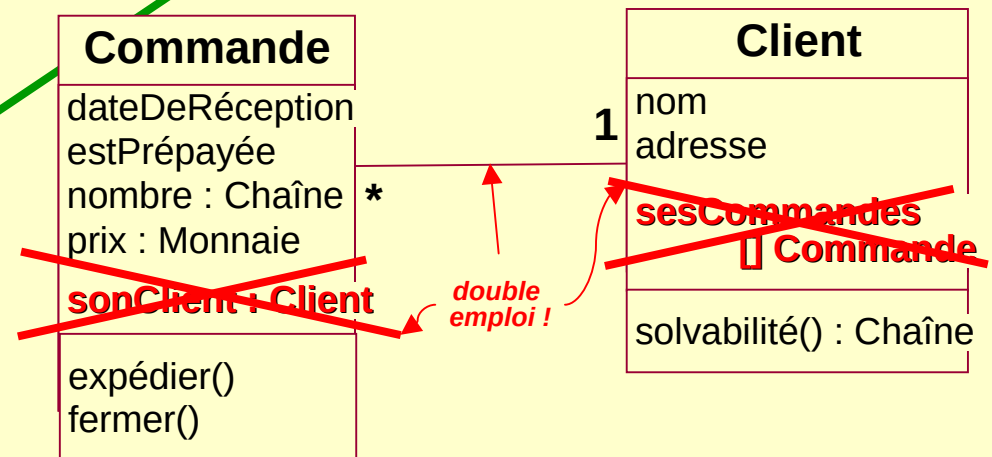
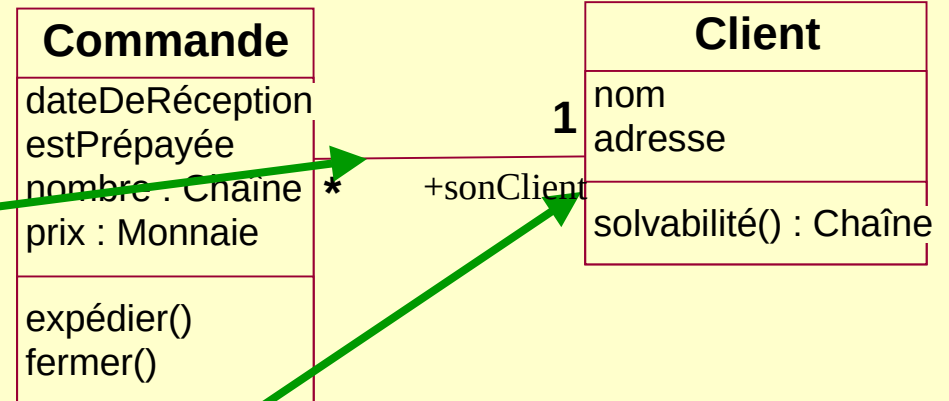
```
    private String dateDeReception;
    private boolean estPrepayee;
    private String nombre;
    private double prix;
    private Client sonClient;
```

```
    public Commande(String date) {...}
    public void expedier() {...}
    public void fermer() {...}
}
```

```
public class Client
{
```

```
    private String nom;
    private String adresse;
    private Commande [] commande;

    public Client(String nom, String adresse) {...}
    public String solvabilite() {...}
}
```



Les concepts : rôles

- Rôles : rôle que joue une classe dans une association :
 - Aux deux extrémités d'une association.
 - Nom de rôle : le rôle est nommé explicitement.
 - Exemple : *ligne* de l'extrémité *Ligne de Commande* associée à *Commande*.
 - En l'absence d'étiquette, le nom de rôle est le nom de la classe cible.

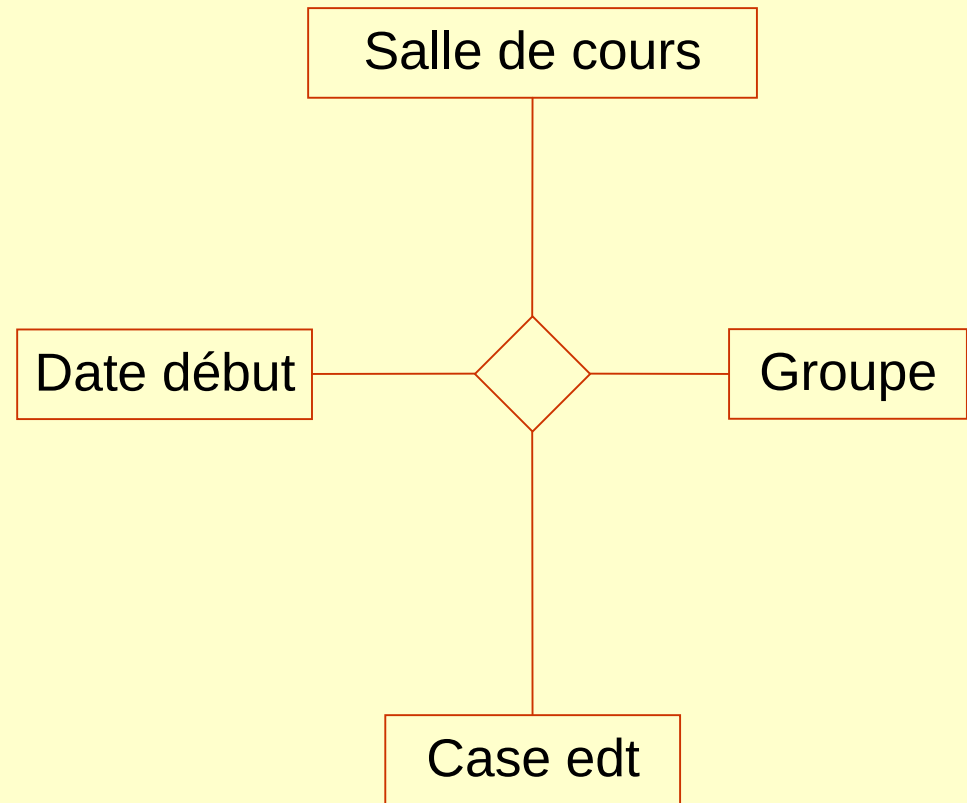
Les concepts : rôle et multiplicités

- Un rôle a une multiplicité :
 - Indique le nombre d'objets susceptibles de participer à une association donnée.
 - Tableau des multiplicités :

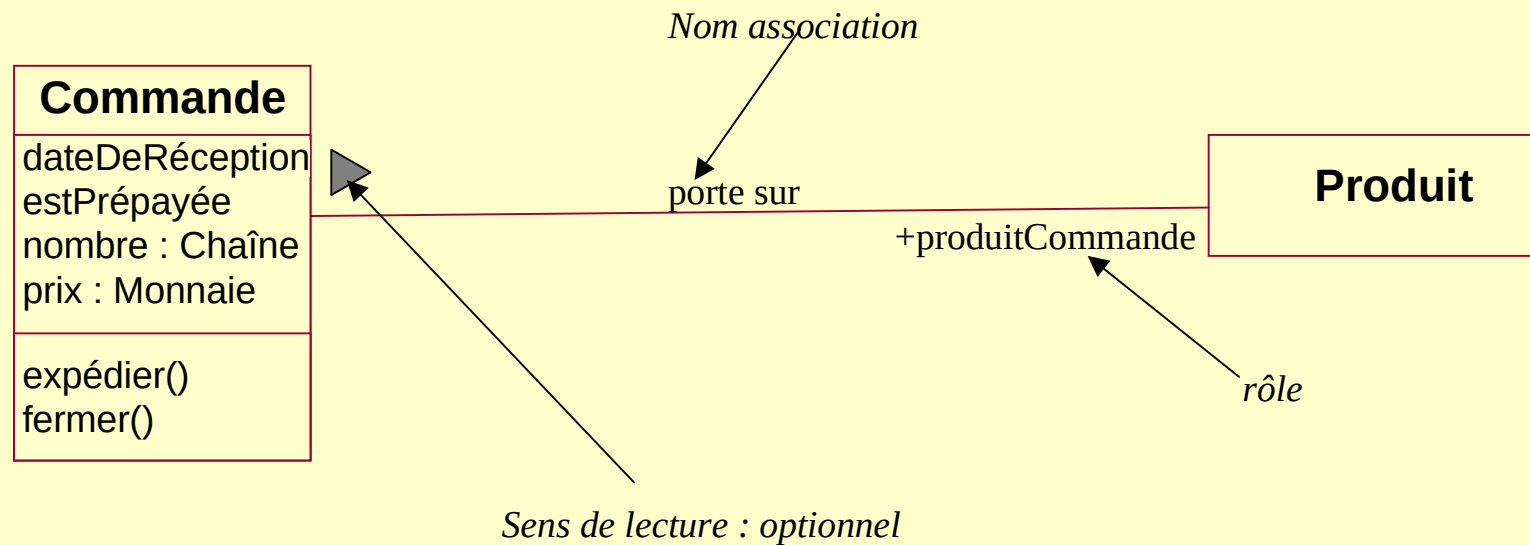
1	Un et un seul
0..1	Zéro ou un
M..N	De M à N
*	De zéro à plusieurs
0..*	De zéro à plusieurs
1..*	D'un à plusieurs
- Une multiplicité peut être :
 - Un nombre : 15 joueurs dans une équipe de rugby
 - Un intervalle : 2..4
 - Une combinaison discrète : 2,4 pour les portes de voitures

Association n-aire

- Porteuse de propriétés
- Fortement déconseillée : se poser des questions !!!!

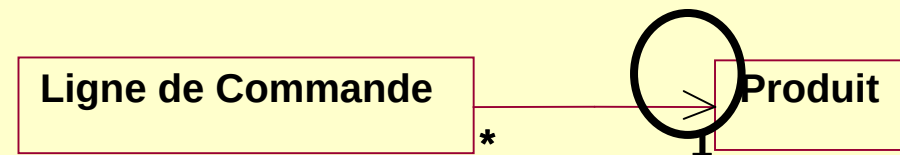


Associations



Associations

- Navigabilité :

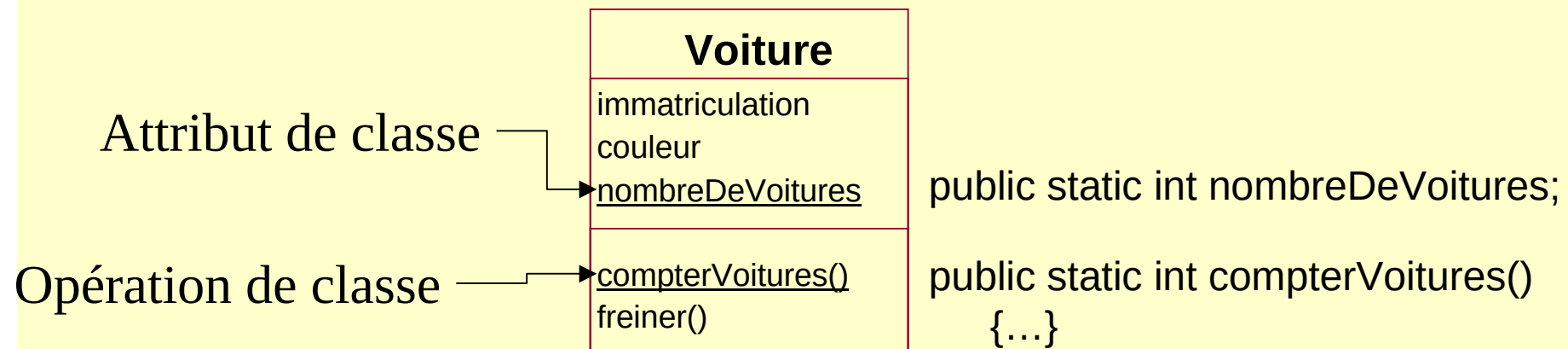


- Association unidirectionnelle.
 - Une Commande peut informer de ses Clients ; mais le client ne peut pas indiquer ses commandes.
- Importance pour l'implémentation et non pour l'aspect purement conceptuel.
 - Commande aura un pointeur sur Client mais Client n'aura pas de pointeurs sur Commande.

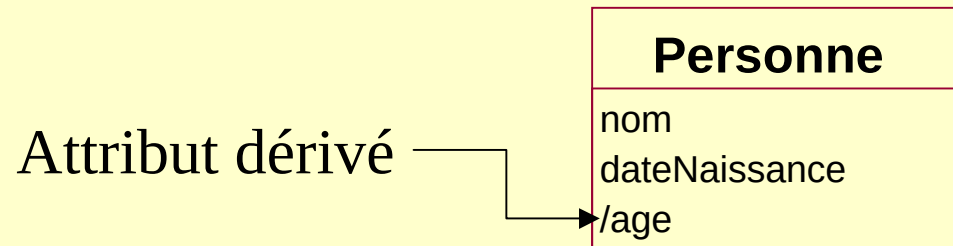
- Association sans navigabilité :

- Bidirectionnelle ou Inconnue.
- Les navigations sont l'inverse l'une de l'autre.

Attributs - opérations de classe



Attributs dérivés



- Attribut dérivé: attribut qui se calcule

Les concepts : attributs

visibilité nom [multiplicité]: type = valeurParDéfaut {propriétés}

+ : publique
: protégée
- : privée

Exemples :

couleurs **[3]**: Color
points **[2]**: Point
nom**[0..1]**: String

{**constante**}

- **Attributs** : exemple : *nom* du client
 - *Niveau conceptuel* : les clients ont des noms
 - *Niveau spécification* : un client peut vous donner son nom et sait le faire.
 - *Niveau implémentation* : Client a un champ (variable d'instance) qui contiendra son nom.
- Plusieurs niveaux de détails :
 - Son nom
 - Son type et sa valeur par défaut

Les concepts : opérations

visibilité nom (liste_paramètres): type_de_retour {propriétés}

+ : publique
: protégée
- : privée

Sens nom_param : type = valeur-défaut

in ← par défaut
out
inout

Exemples:

{requête}
{constructeur}

- Opérations : principales actions de la classe
 - Processus qu'une classe sait mener à bien.
 - Correspond aux méthodes implémentées d'une classe.
 - Au niveau conceptuel, on ne spécifie généralement pas les opérations de manipulation des attributs. Elles sont généralement déduites.

Les concepts : opérations

- Une opération est invoquée sur un objet.
- Une méthode est le corps d'une opération.
- Différence pertinente pour le polymorphisme :
 - Une opération au sein d'une classe.
 - Autant de méthodes qui redéfinissent cette opération dans les sous-classes.
- Nécessité de faire la distinction dans de telles situations.

Quelques précisions

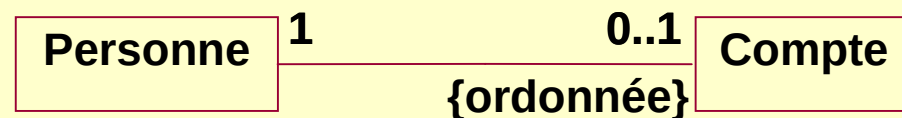
- L'exemple des commandes laisse entendre :
 - Qu'il y a une ou plusieurs méthodes associées à Client qui indiqueront quelles Commandes un client a passées.
 - Qu'une commande a des méthodes qui informeront du Client ayant passé la Commande et ses Lignes de Commande.
- Absentes du diagramme de classes :
 - Méthodes de requêtes.
 - Liées généralement aux choix de l'implémentation.
- Pas de détails d'implémentation tels que les structures de données.
- Pas de détails d'implémentation des associations (pointeurs ou collections d'autres objets).

Contraintes

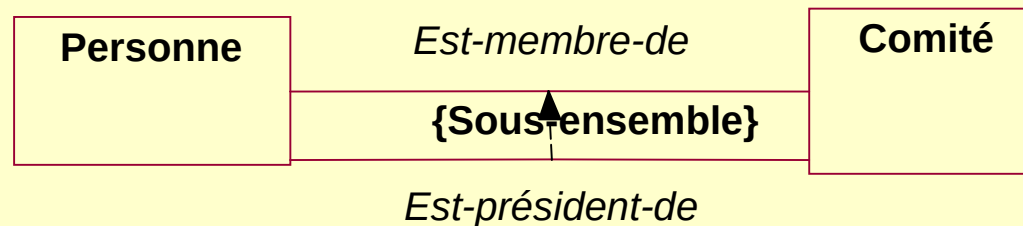
- Associations, attributs et généralisation :
 - Expriment les contraintes importantes
 - Ne peuvent pas toutes les indiquer. Il faut cependant les capturer. UML fait le choix de les exprimer au niveau du diagramme de classes.
- Contraintes :
 - Exprimées entre accolades.
 - En langage naturel ou en OCL (*Object Constraint Language*).
 - Généralement implémentées sous forme d'assertions dans le langage de programmation.

Quelques contraintes

- Contrainte d'ordre sur les associations : {ordonnée}
 - Placée sur le rôle. Spécifie qu'une relation d'ordre décrit les objets de la collection.

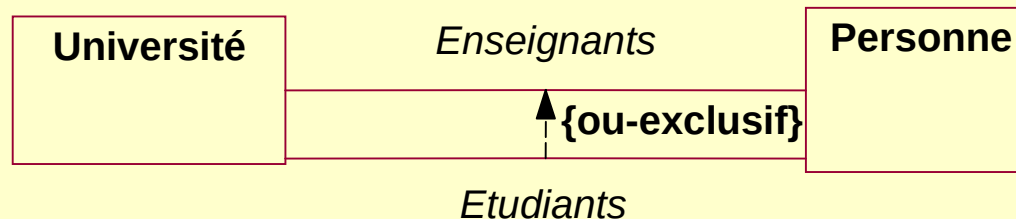


- Contrainte de sous-ensemble : {sous-ensemble}
 - Une collection est incluse dans une autre collection.



Quelques contraintes

- Contrainte du ou-exclusif : {ou-exclusif}
 - Pour un objet, une seule association valide parmi un groupe d'associations.



- Évite l'introduction de sous-classes artificielles pour représenter l'exclusivité.

Généralisation

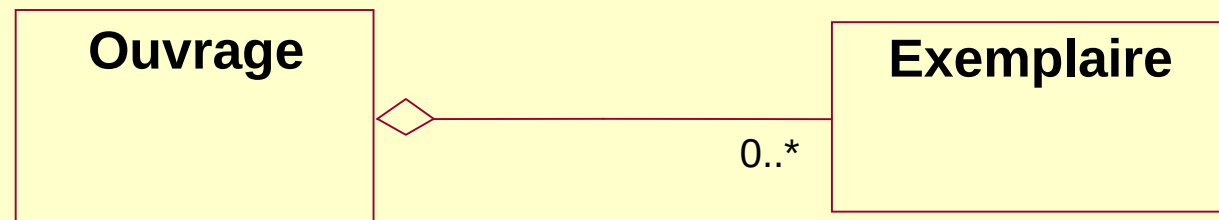
- Généralisation :
 - Relation non-symétrique.
 - Relation entre une classe et une ou plusieurs de ses versions plus raffinées ou plus spécialisées.
 - Relation de classification entre un élément plus général et un élément plus spécifique.
- Héritage :
 - Mécanisme objet qui permet aux classes de partager des attributs et des opérations en se basant sur une relation, habituellement la généralisation.

Super-classe et Sous-classe

- Super-classe :
 - Contient les attributs et les opérations communs.
- Sous-classe :
 - Raffine ou restreint les attributs et opérations hérités.
 - Ajoute ses propres attributs et opérations.
- Une sous-classe **hérite** de sa super-classe :
 - Attributs
 - Opérations
 - Associations
- Héritage multiple :
 - Une classe hérite de ses super-classes.
 - Ses attributs et opérations sont ceux de ses parents.
 - UML ne fournit pas de règles de résolution de conflits.

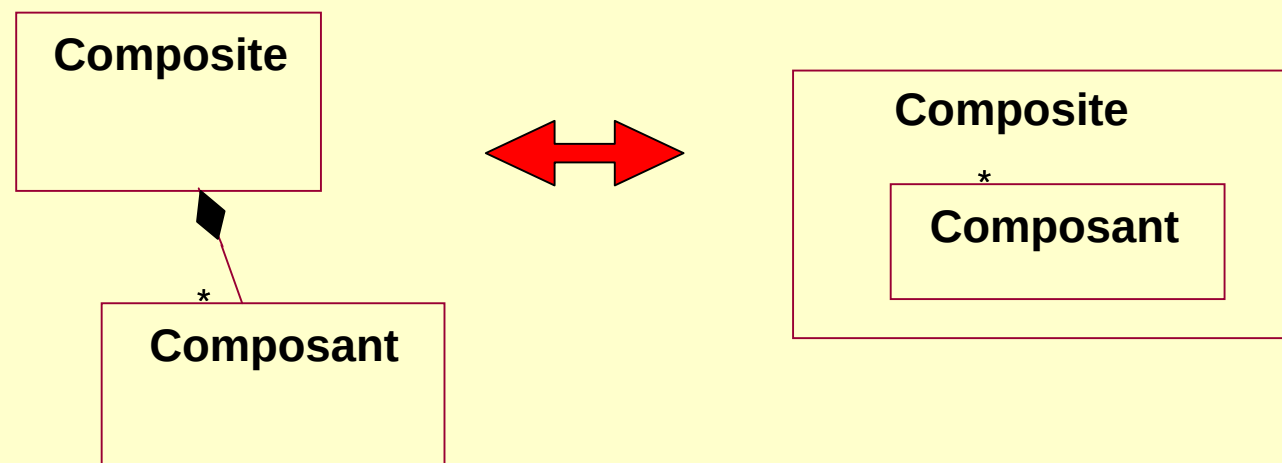
Agrégation

- Association :
 - Couplage faible
 - Relative indépendance entre classes.
- Agrégation : association entre classes :
 - Forme d'association ayant un plus fort couplage.
 - Connexions bidirectionnelles et dissymétriques.
 - Exprime souvent un fort couplage logique.
 - Relation transitive, non symétrique et réflexive.
 - Exemple : un ouvrage (*agrégat*) et ses exemplaires (*composant*).



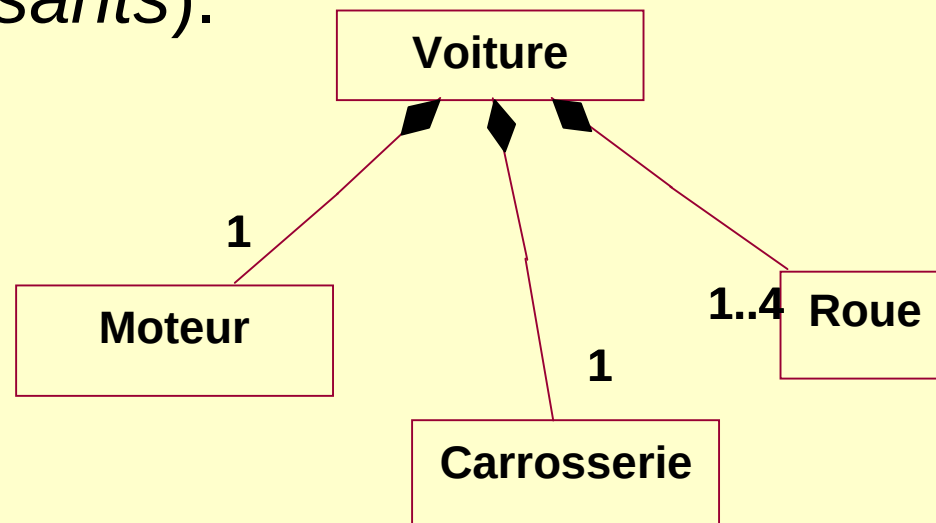
Composition

- Composition :
 - Forme d'agrégation ayant un plus fort couplage.
 - Exprime souvent une composition physique.
 - UML offre deux possibilités de représentation :

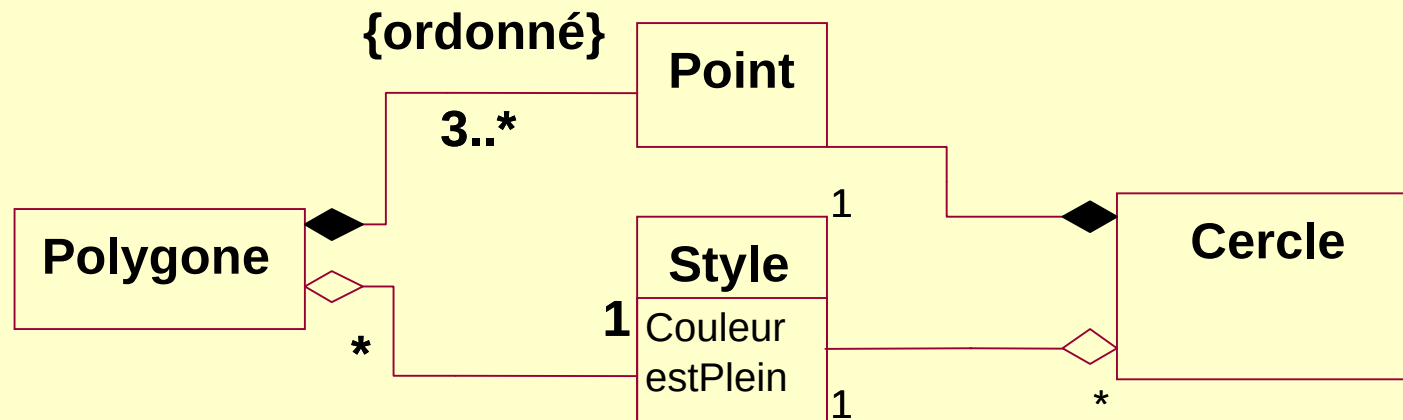


Composition

- Composition :
 - Exemple : une voiture (*composite*) et son moteur, sa carrosserie et ses roues (*composants*).



Agrégation et Composition



- un Point et un Cercle sont composés de points :
 - la destruction du composite implique la destruction du composant
- un Style peut concerner plusieurs Polygones et/ou Cercles :
 - Style est indépendant de l'existence des agrégats
 - Sa modification implique forcément celle de ses agrégats.

Quand les utiliser ?

- Diagrammes de classes :
 - Épine dorsale.
 - Riche en notations.
- Ne pas chercher à utiliser toutes les notations.
- Commencer par les concepts simples et indispensables : classes, attributs, associations, généralisation et contraintes.
- Ne pas créer systématiquement des diagrammes :
 - Peu de diagrammes explicites, faciles à utiliser et à mettre à jour,
 - Qu'un foisonnement de diagrammes obsolètes.
- Ne PAS s'enliser dans des détails d'implémentation.

Stéréotype

- Principal mécanisme d'extension d'UML.
- Permet de construire de nouveaux types, par exemple des objets affichables.
- Généralement représenté par un texte entre guillemets (exemple <<objetAffichables>>) ou de définir une icône.
- Stéréotypes de classes, d'association ou de généralisation.

Qualification

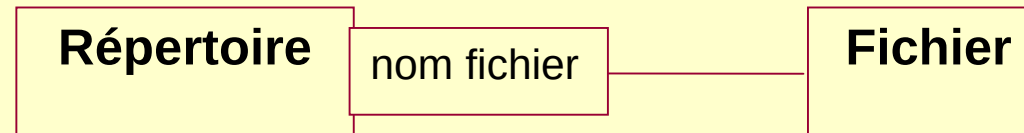
- Une association peut être qualifiée.
- Restriction qui consiste à sélectionner un sous-ensemble d'objets parmi l'ensemble des objets qui participent à une association.
- La paire (instance de la classe A, valeur de la qualification) identifie un sous-ensemble des instances de la classe B.



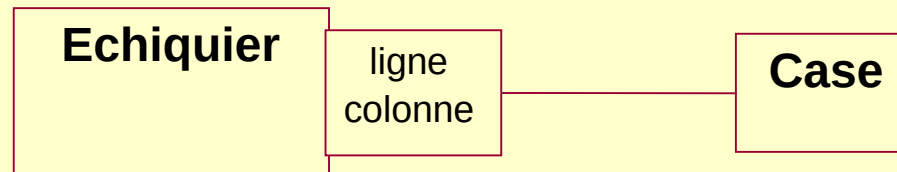
- La clé est représentée sur le rôle de la classe de départ, dans un rectangle.

Exemples de qualification

- Pour un répertoire donné, un fichier est identifié par son nom :



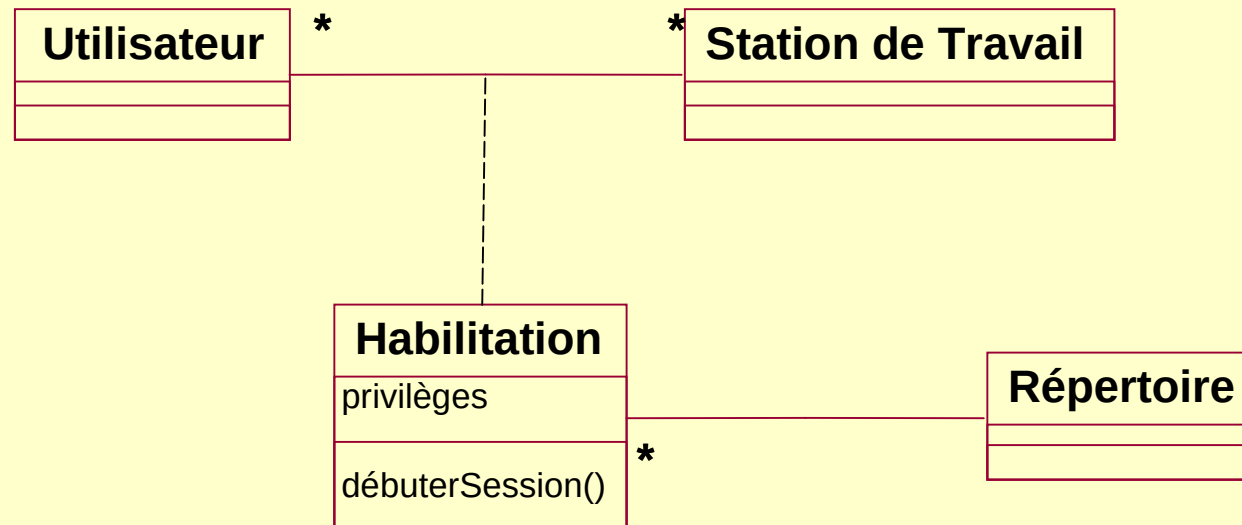
- Un qualificatif peut être composé :



- Combinaison d'une ligne et d'une colonne pour identifier une case de l'échiquier.

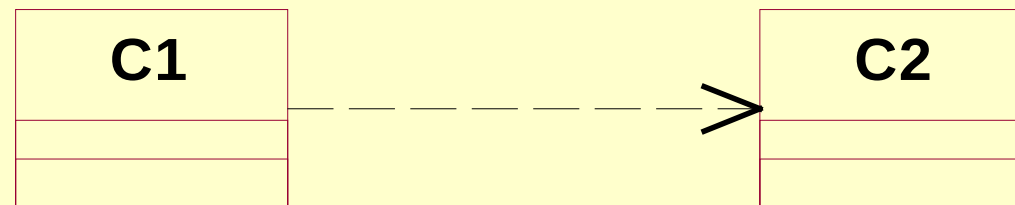
Classe-association

- Une association peut être représentée par une classe si elle est porteuse d'attributs/opérations.
- C'est une classe comme les autres. Elle peut avoir des relations.



Dépendance entre classes

- L'utilisation dans une méthode d'une classe C1 d'une variable locale ou d'un paramètre formel qui sont de type C2, C2 étant une classe, se note :

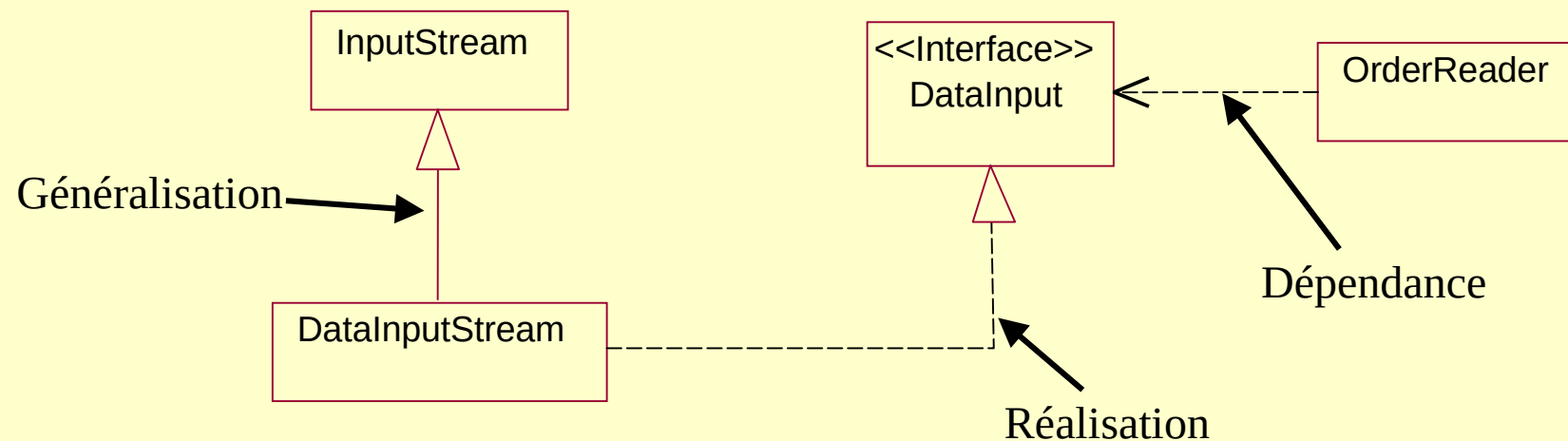


Interfaces /Classes abstraites

- Classe abstraire :
 - son interface est la signatures de toutes ses propriétés non privées.
 - Notation : nom en italique ou la contrainte {abstract}
- Interface :
 - Utilise un type pour décrire le comportement visible d'une classe, sans implémentation.
 - Peut ainsi fournir une vue partielle ou globale d'un ensemble de services.
 - Est un stéréotype.
- Une classe peut :
 - Spécialiser une classe abstraite.
 - Réaliser une interface.

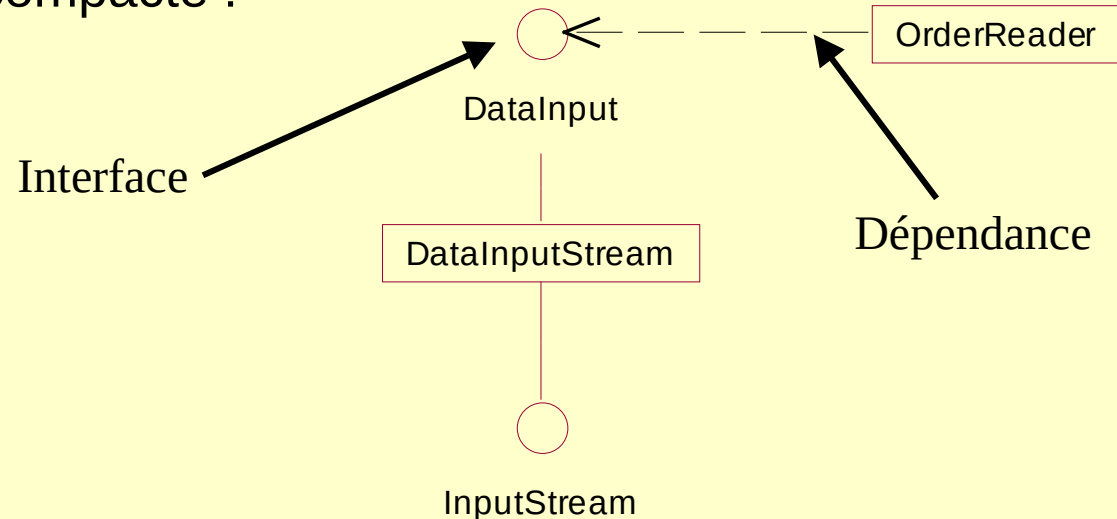
Interfaces - Classes abstraites

- En UML, le symbole de la réalisation est délibérément semblable à la généralisation.
- Réalisation : une classe implémente le comportement spécifié par une autre. Une classe d'implémentation peut en réaliser une autre : elle se conforme à son interface sans en hériter forcément.



Interfaces - Classes abstraites

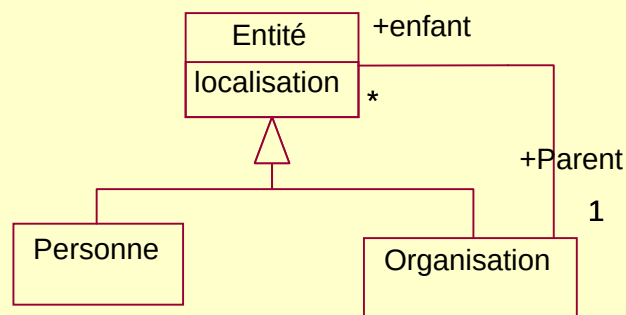
- Autre notation plus compacte :



- la réalisation des interfaces est représentée par de petits cercles, rattachés à la classe qui réalise l'interface.
- Ne distingue pas entre réalisation d'interface et sous-classement de classe abstraite.
- Mais ne permet pas de représenter les opérations des interfaces ni les relations de généralisation entre elles.

Diagramme d'objets

- Un *diagramme d'objets* ou *d'instances* est un instantané des objets présents à un moment donné.
- Diagramme de collaborations dépourvu de messages.
- Utilité : pour représenter une configuration particulière d'objets, surtout quand les liens possibles entre objets sont complexes :



Structure d'une entité

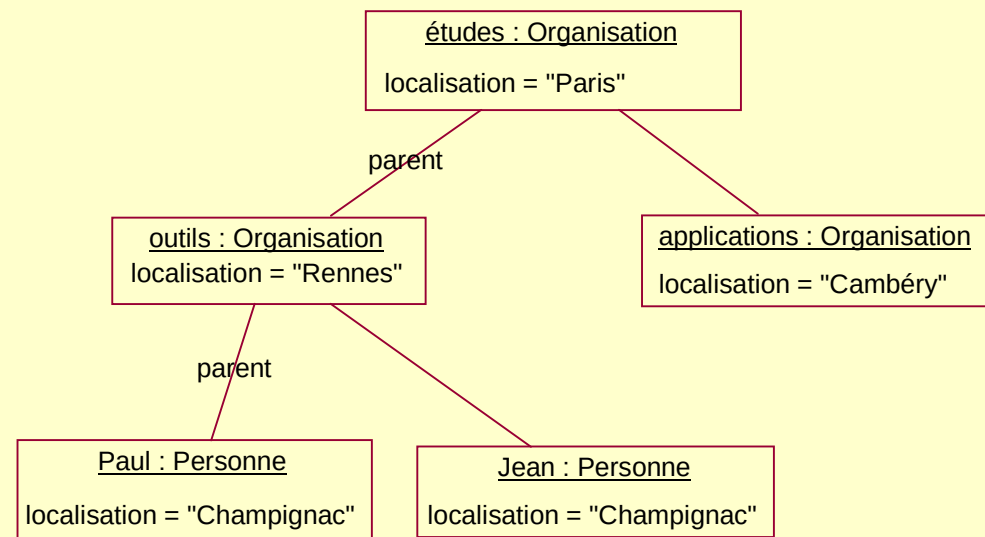


Diagramme d'objets : exemples d'instances d'une entité

Diagrammes d'Interactions

- *de séquence*
- *de collaboration*

Diagrammes d'interactions

- Modèles :
 - manière dont des groupes d'objets collaborent
 - pour réaliser un comportement donné.
- Un diagramme d'interaction typique :
 - Capture le comportement d'un seul cas d'utilisation.
 - Représente pour la réalisation d'un cas d'utilisation:
 - un certain nombre d'objets
 - les messages transmis entre ces objets.

Diagrammes d'interactions

- Deux sortes de diagrammes d'interactions :
 - les diagrammes de séquence
 - les diagrammes de collaboration

Diagramme de séquences

- Description d'un cas d'utilisation
- Selon un aspect temporel
 - étapes
 - enchaînements
 - exécutions en parallèle

Un exemple ...

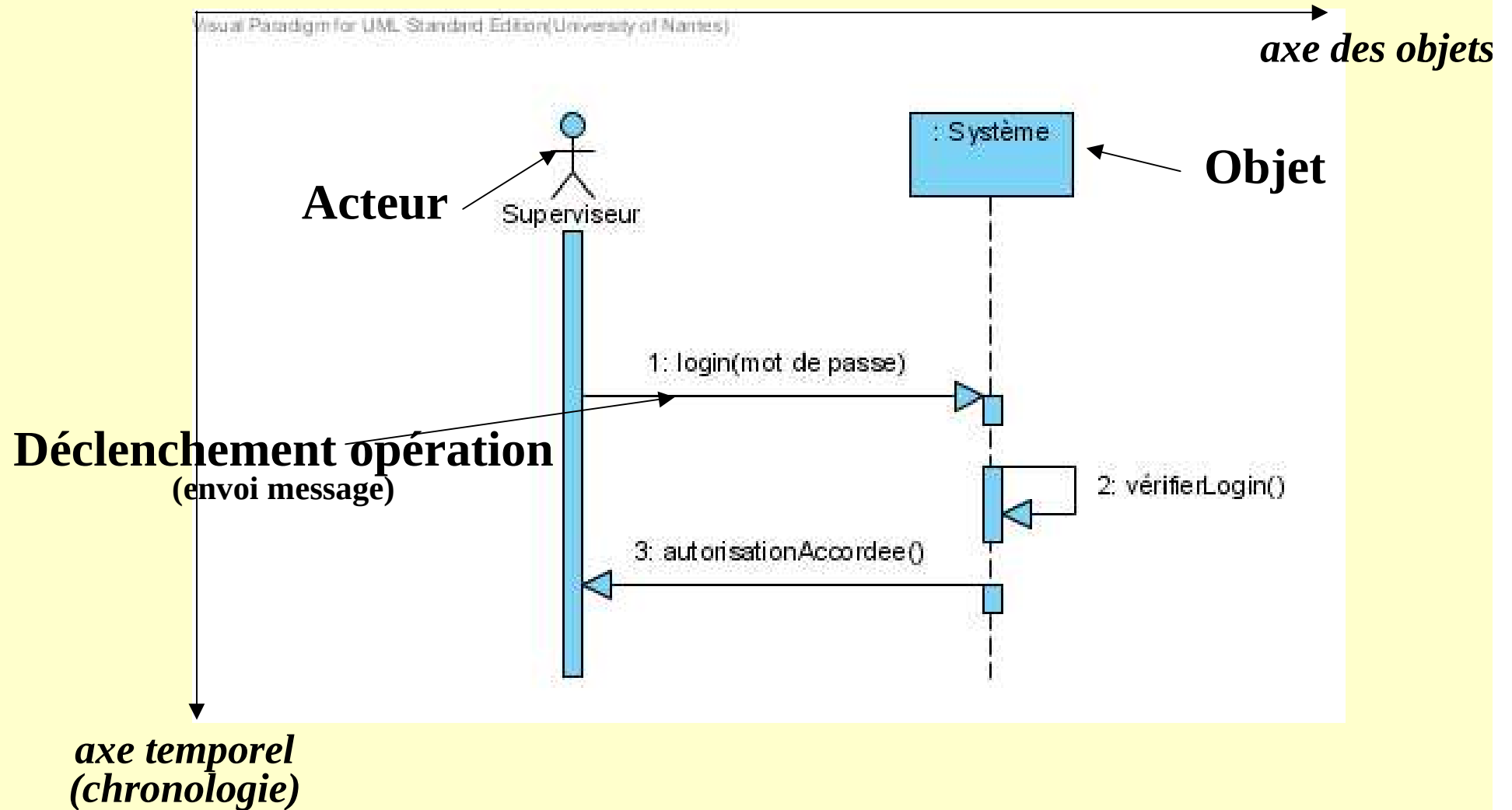


Diagramme de séquence

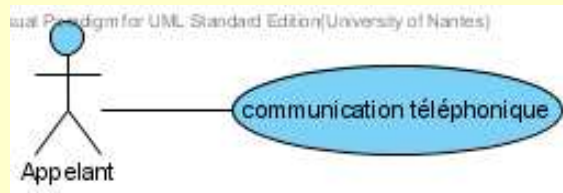
- Décrit comment les objets interagissent au sein du système, au vu des fonctionnalités.
- Vue graphique d'un scénario :
 - Mettant en évidence les interactions d'objets
 - Selon un point de vue temporel
- Généralement associés aux cas d'utilisation.
- Etabli le rôle des objets et aide à cerner les responsabilités des classes et interfaces.
- A deux dimensions :
 - verticale : représente le temps
 - horizontale : représente les différents objets.

Deux utilisations possibles

- Documentation des cas d'utilisation :
 - description de l'interaction entre les entités
 - sans rentrer dans le détail d'implémentation ni de la synchronisation.
 - les événements surviennent dans le domaine de l'application.
- Usage détaillé :
 - description des interactions entre objets, instances du diagramme de classes.
 - ces interactions sont des envois de messages, en déclenchant des méthodes d'instances (ou de classes).

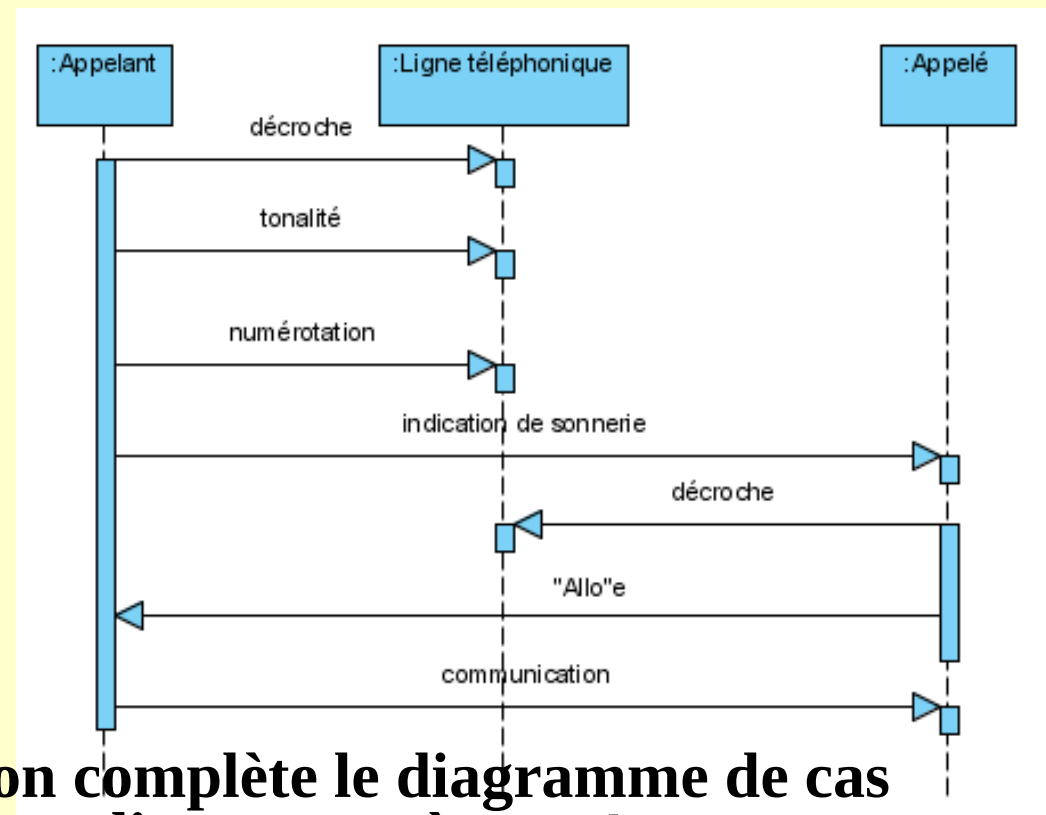
Exemple d'utilisation

- Documentation de cas d'utilisation ⁽¹⁾ : communication téléphonique.



Un cas d'utilisation

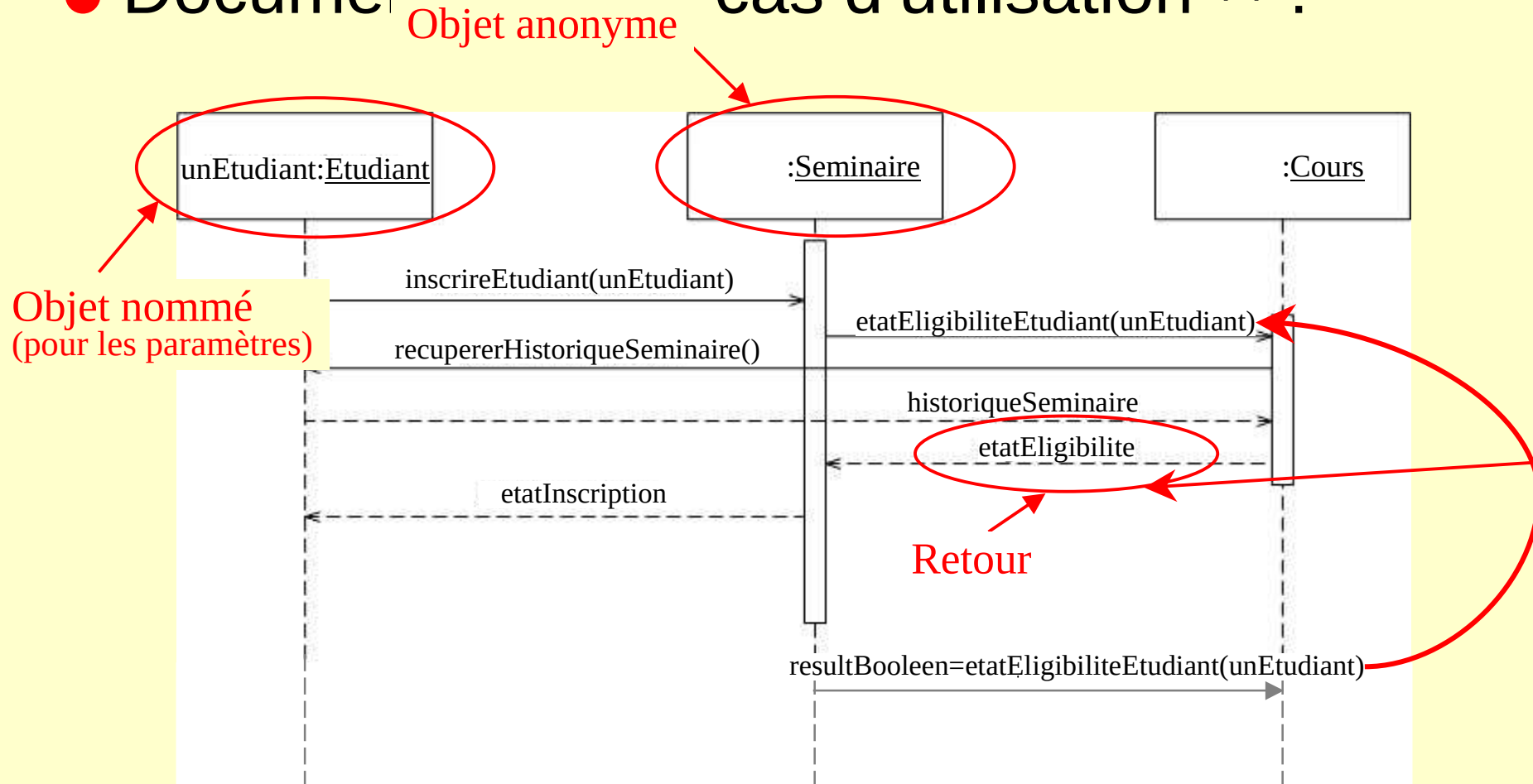
*Sa description
(informelle)*



Attention ! Cette description complète le diagramme de cas d'utilisation ; ce n'est pas un diagramme à part !

Exemple d'utilisation

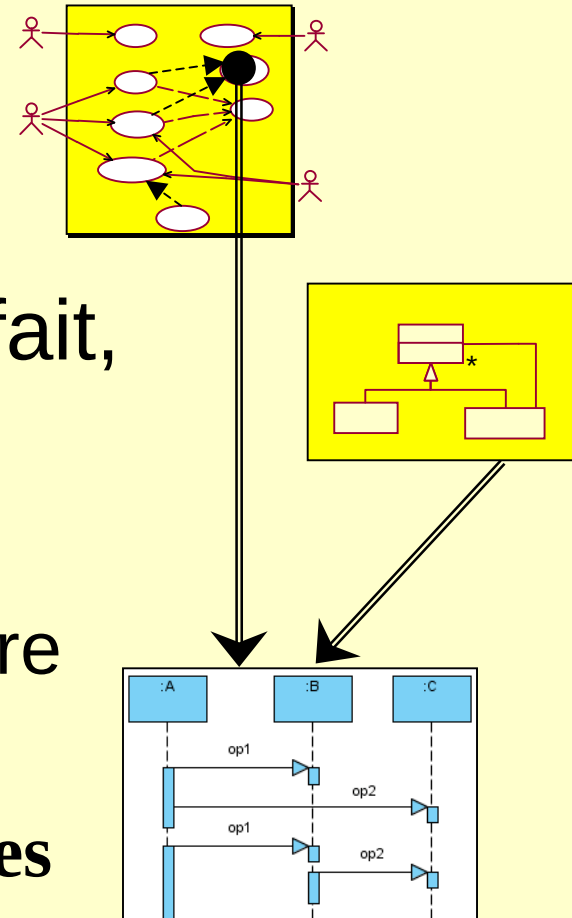
- Documentation de cas d'utilisation (2) :



Interactions entre objets

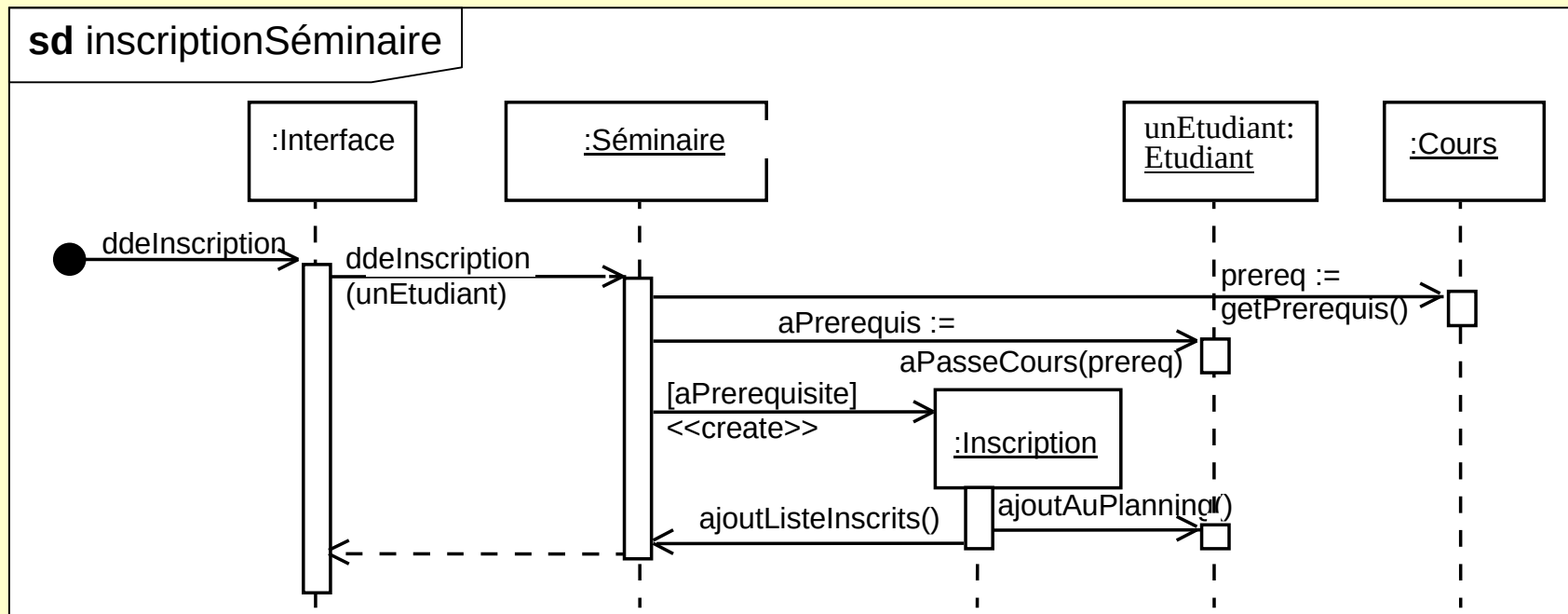
- Une fois le diagramme de cas d'utilisation établi et ses scénarios décrits
- ET le diagramme de classes fait,
- pour chaque cas d'utilisation,
 - quels sont les objets
 - qui interagissent pour y répondre

Diagramme de séquences

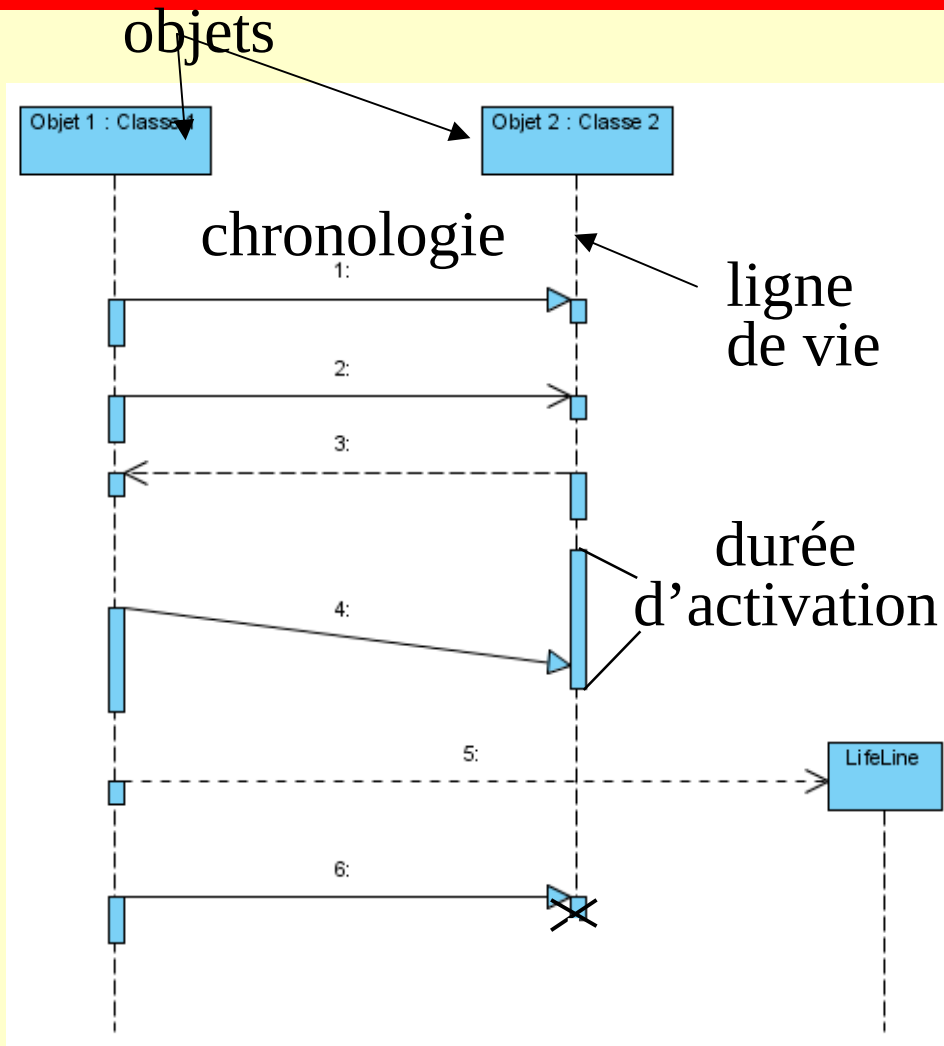


Exemple

- Description cas d'utilisation tp. 98 a pour diagramme de séquences



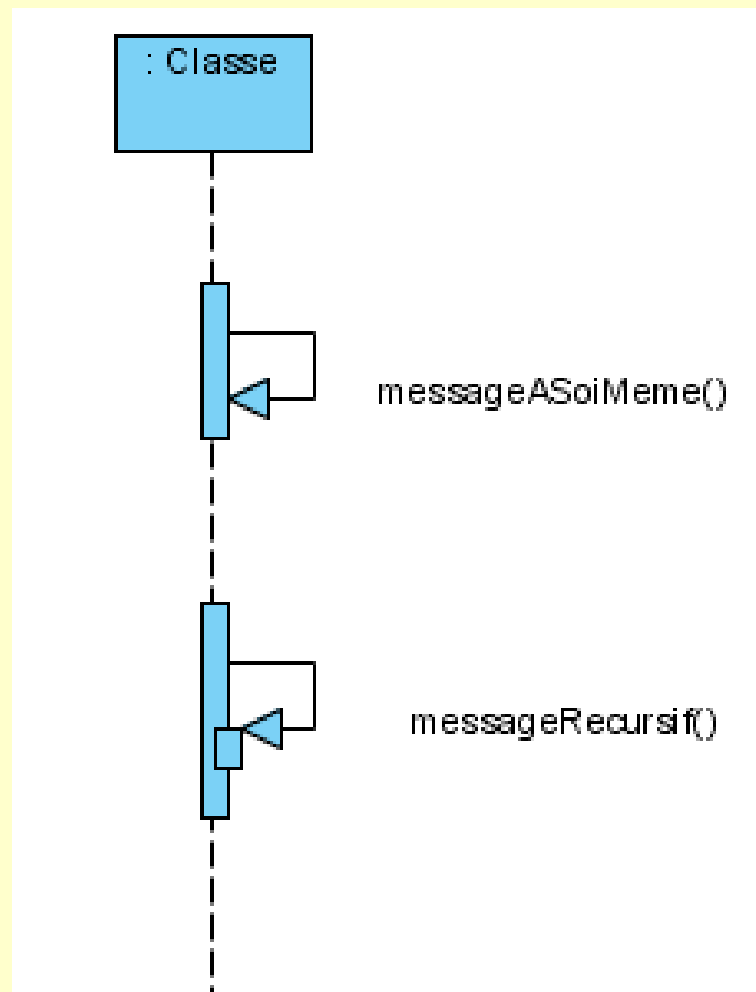
Formalisme graphique



Objet 1 de Classe 1
Objet 2 de Classe 2
interagissent

- Message synchrone
- Message asynchrone
- Retour de message
(optionnel pour message synchrone)
- Délai de propagation
- Création d'un objet (*new*)
- Destruction de l'objet

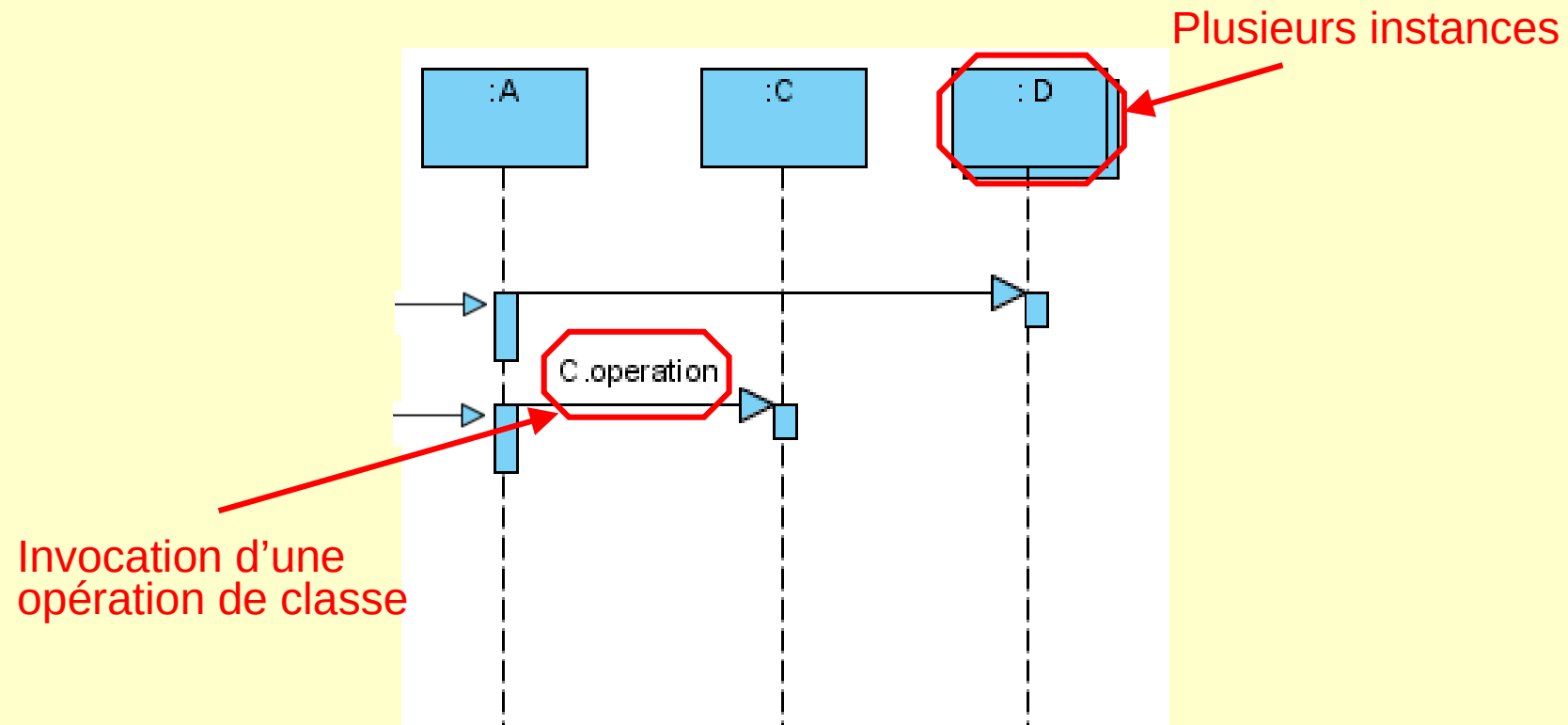
Formalisme graphique



Notions et concepts

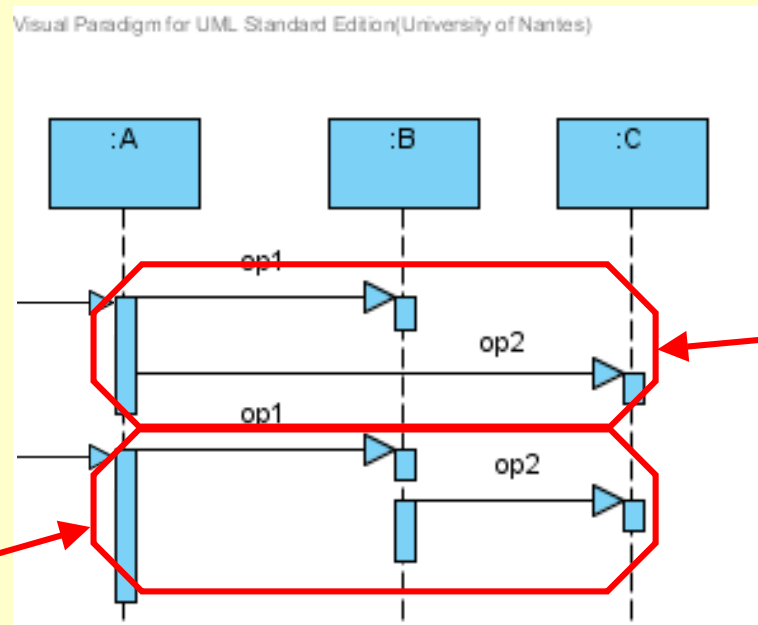
- Ligne de vie de l'objet : vie de l'objet durant l'interaction.
- Message entre deux objets :
 - porte au moins son nom et les paramètres ainsi que des structures de contrôle.
 - flèche entre les deux lignes de vie : de l'objet émetteur vers l'objet receveur.
- Ordre chronologique des messages : de haut en bas.
- Objet est actif :
 - sa période d'activité : boîte verticale d'activation sur la ligne de vie.
 - toute activation durant cette période y est incluse.
- Condition :
 - si simple : sous forme de texte
 - si complexe : développer par un diagramme de séquence.

Formalisme graphique : suite



Structures de contrôle

- Les formes des diagrammes de séquence reflètent indirectement les choix de structures :

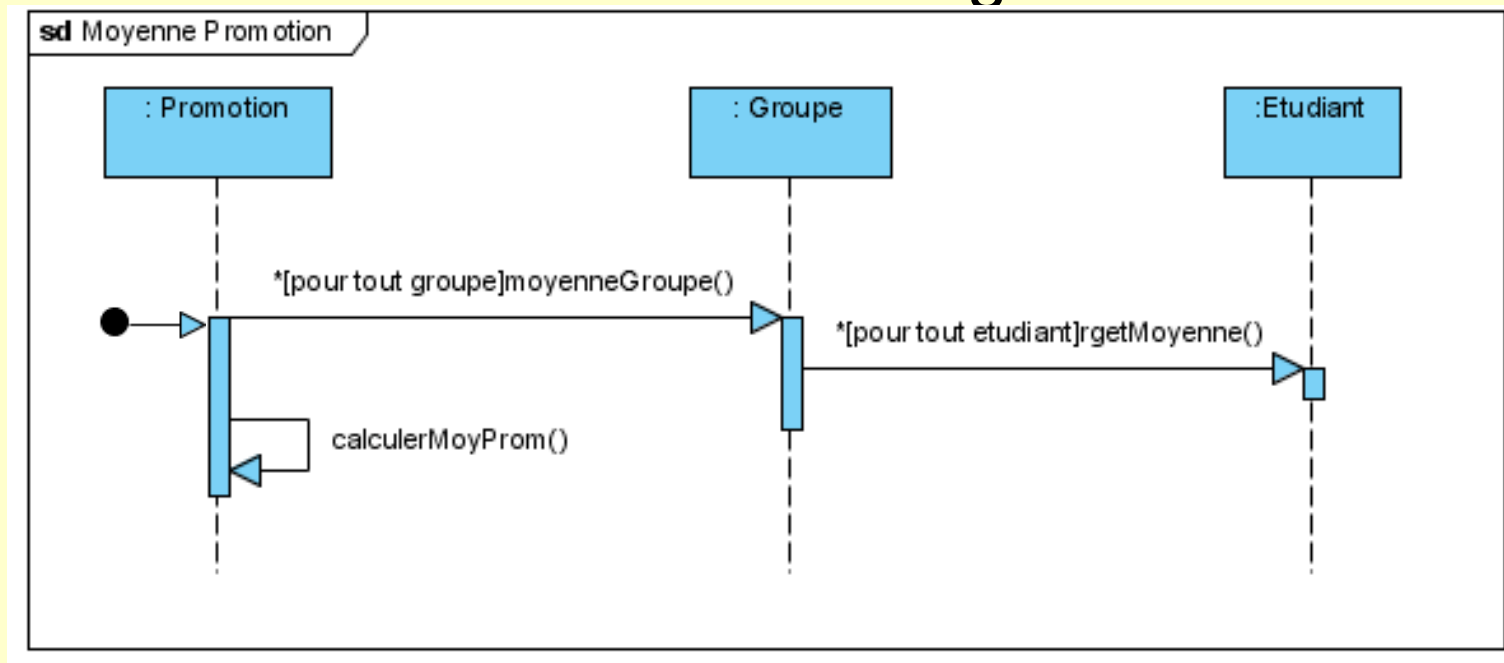


Contrôle centralisé

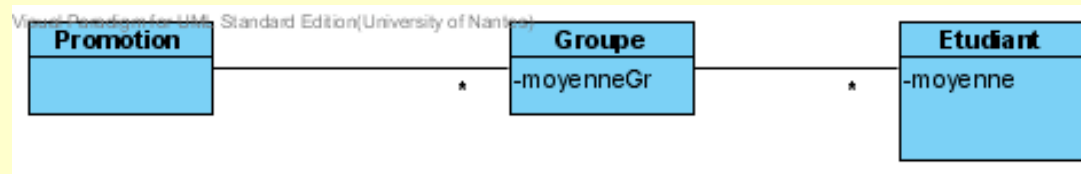
Contrôle décentralisé

Structures de contrôle

- L'itération d'un envoi de message : *

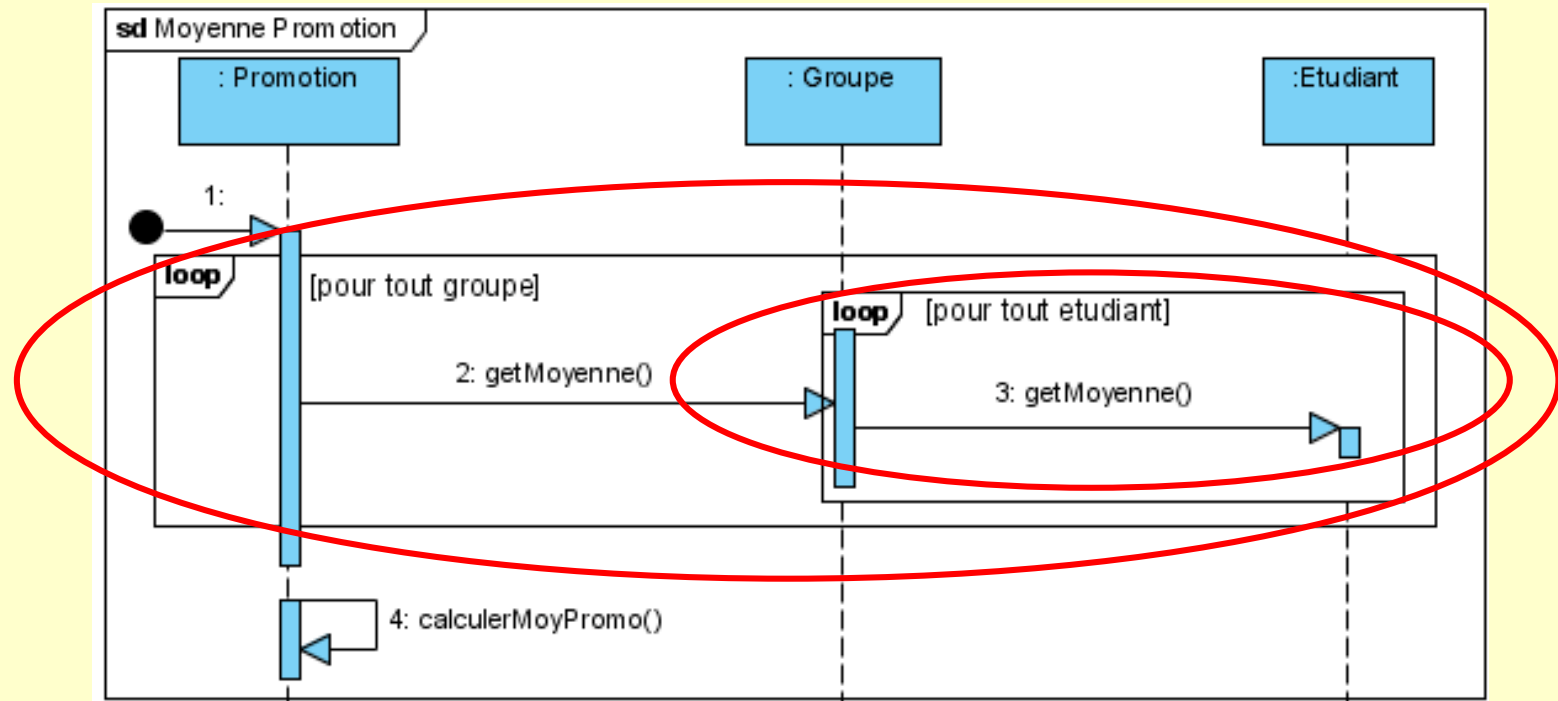


- Envoi de message s'appuie sur une association /dépendance



Structures de contrôle

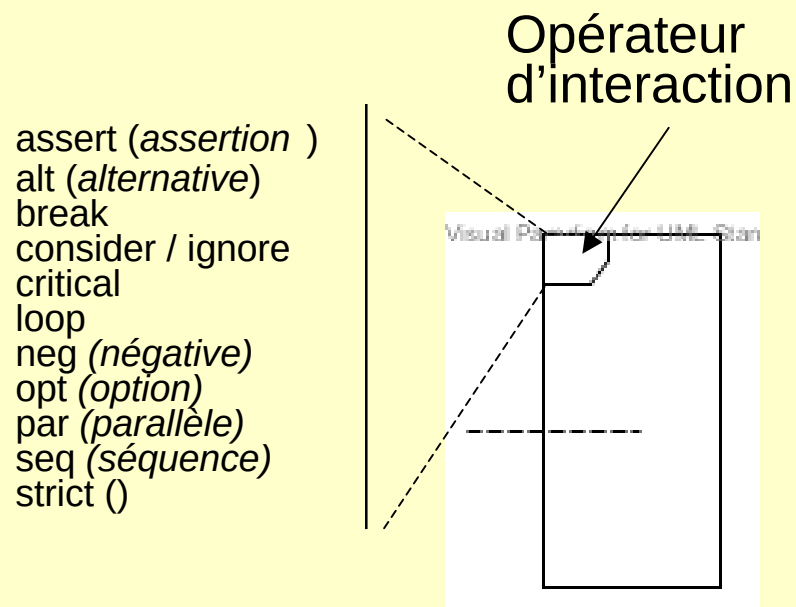
- L'itération d'un envoi de message : *loop*



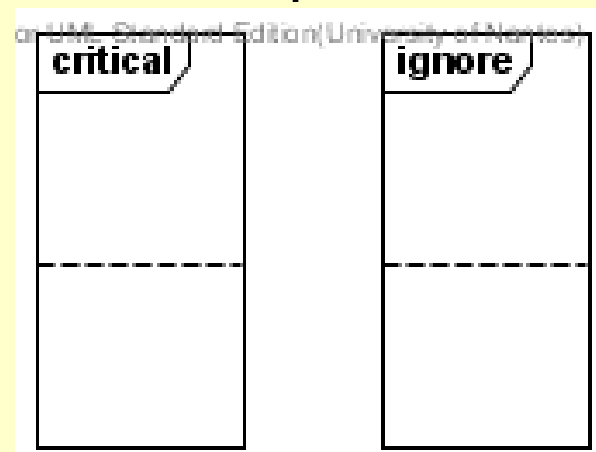
Utilisation de *fragments combinés*

Fragments combinés

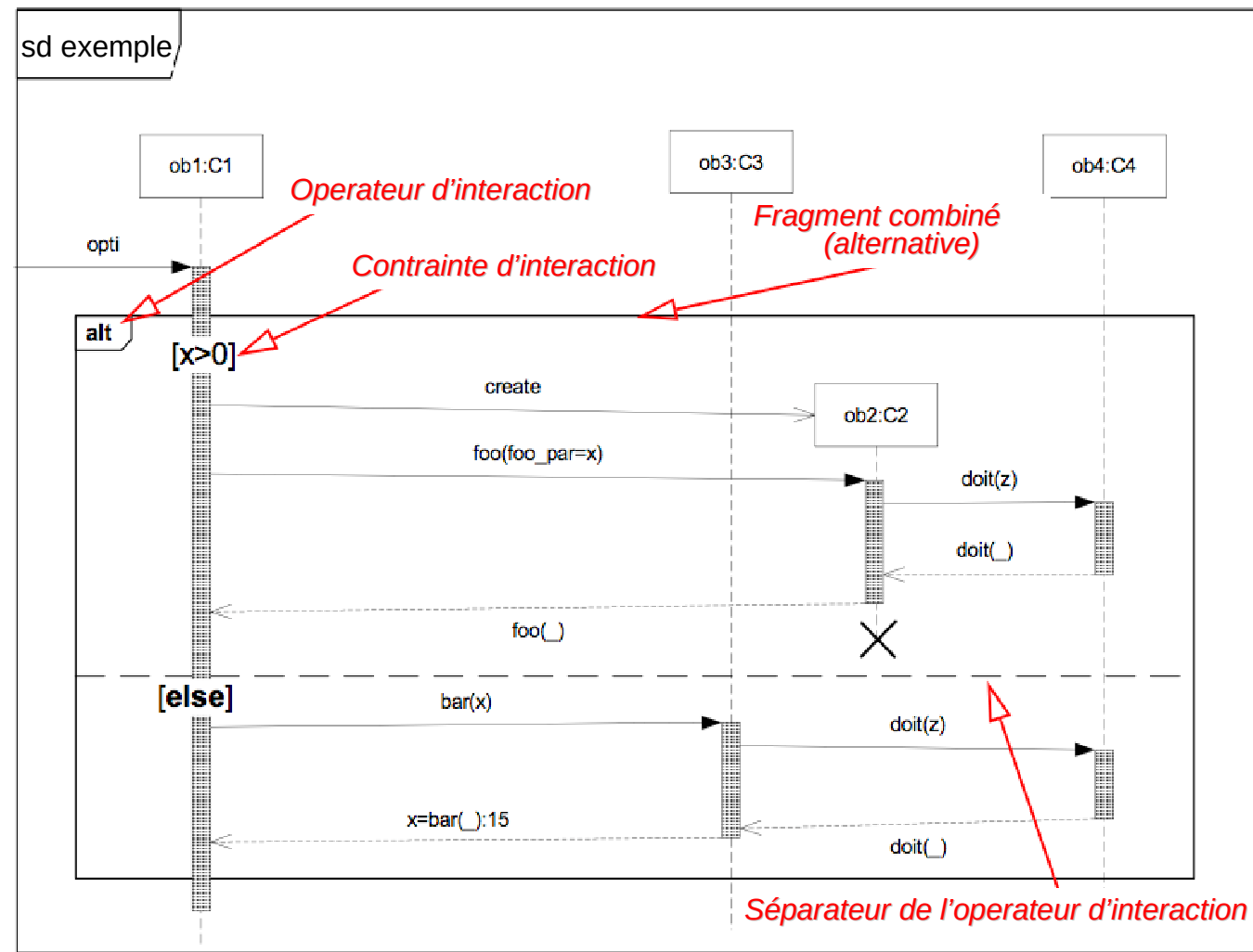
- Articulation d'interactions
- Description compacte des diagrammes de séquences
- Représentation de l'ensemble des entités participantes ou une partie



Exemple :



Exemple



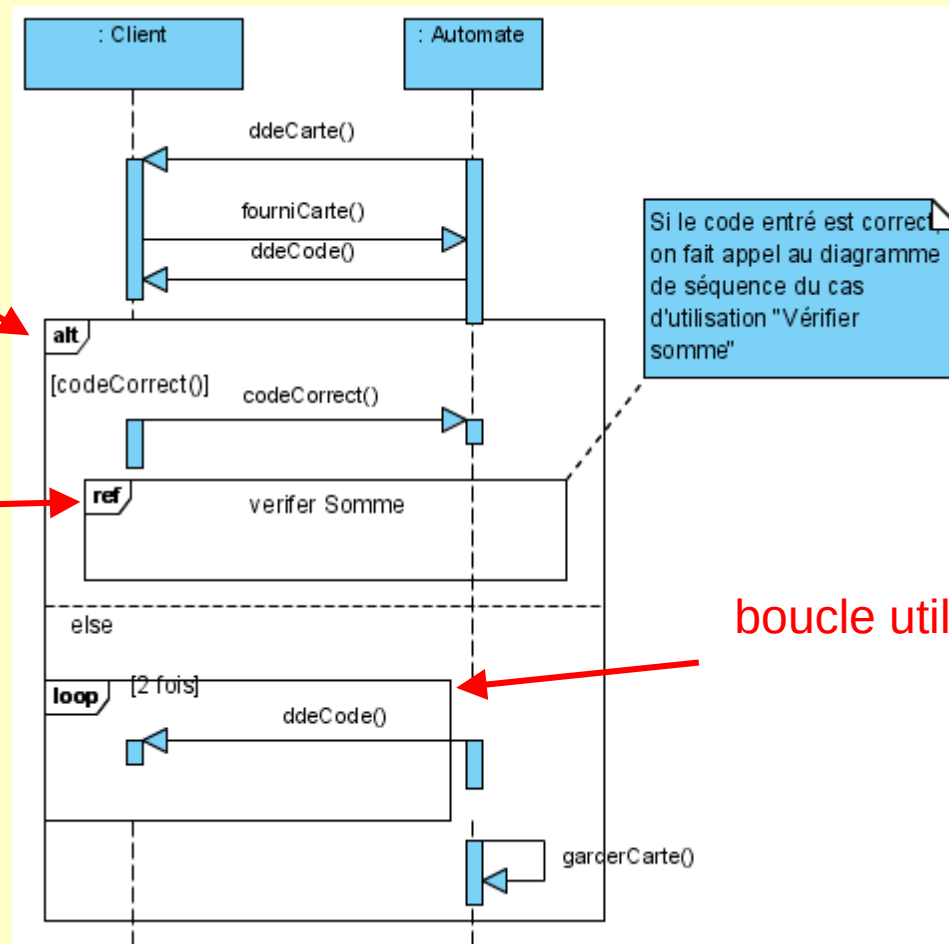
Fragments combinés *alt* – *ref* – *loop*

équivalent du
« *si – alors – sinon* »

alternative

appel à un
autre
diagramme de
séquences
(*interaction use*)

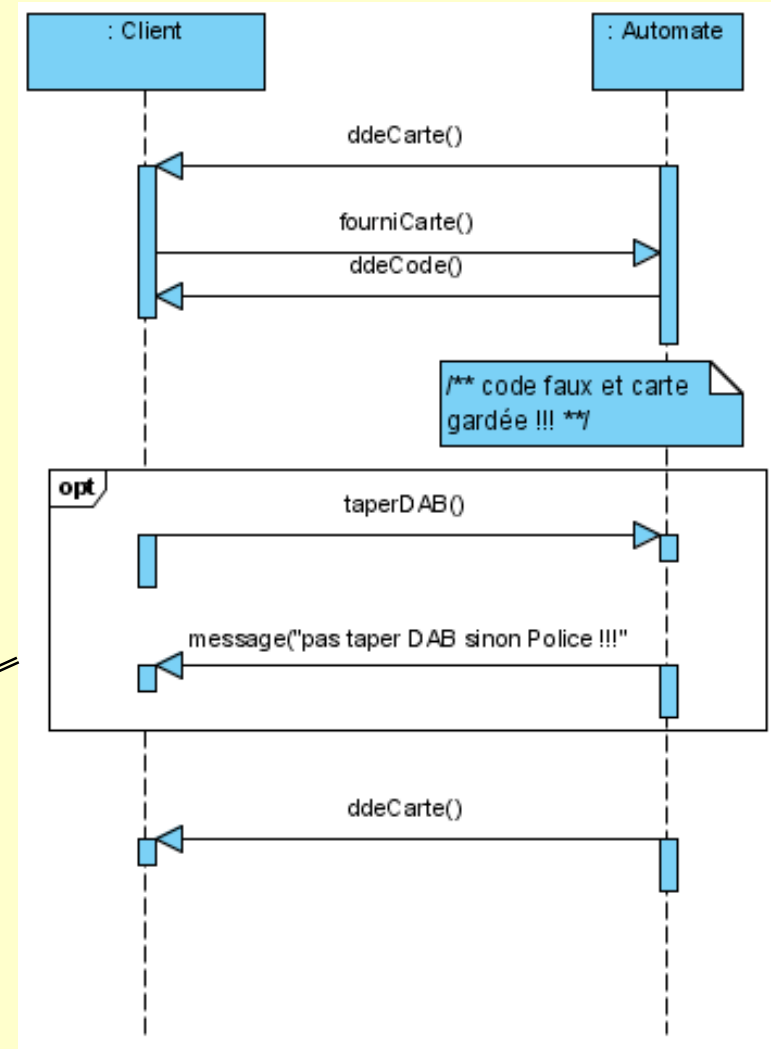
Lien présent dans le
diagramme de cas
d'utilisation (« *uses* »
dans ce cas)



Fragments combinés *opt*

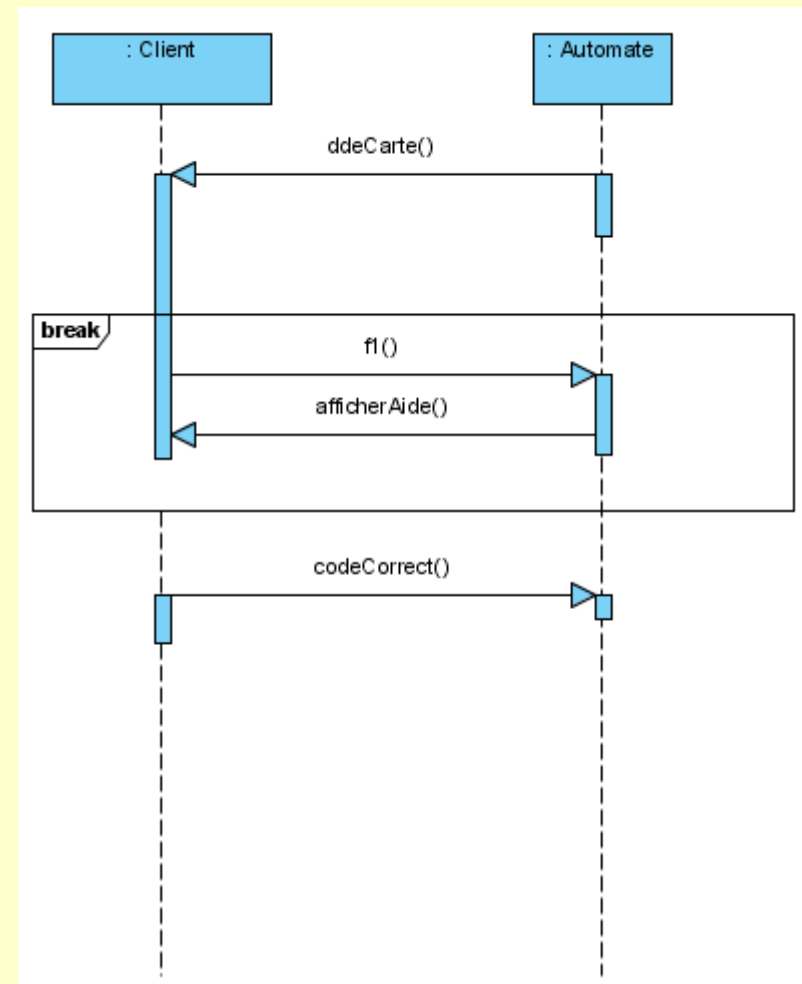
- un comportement optionnel :
- *Rares sont les utilisateurs mécontents qui se défoulent sur le DAB ... mais il peut y en avoir ! ... Que faire dans ce cas ?*

équivalent du
« si – alors »



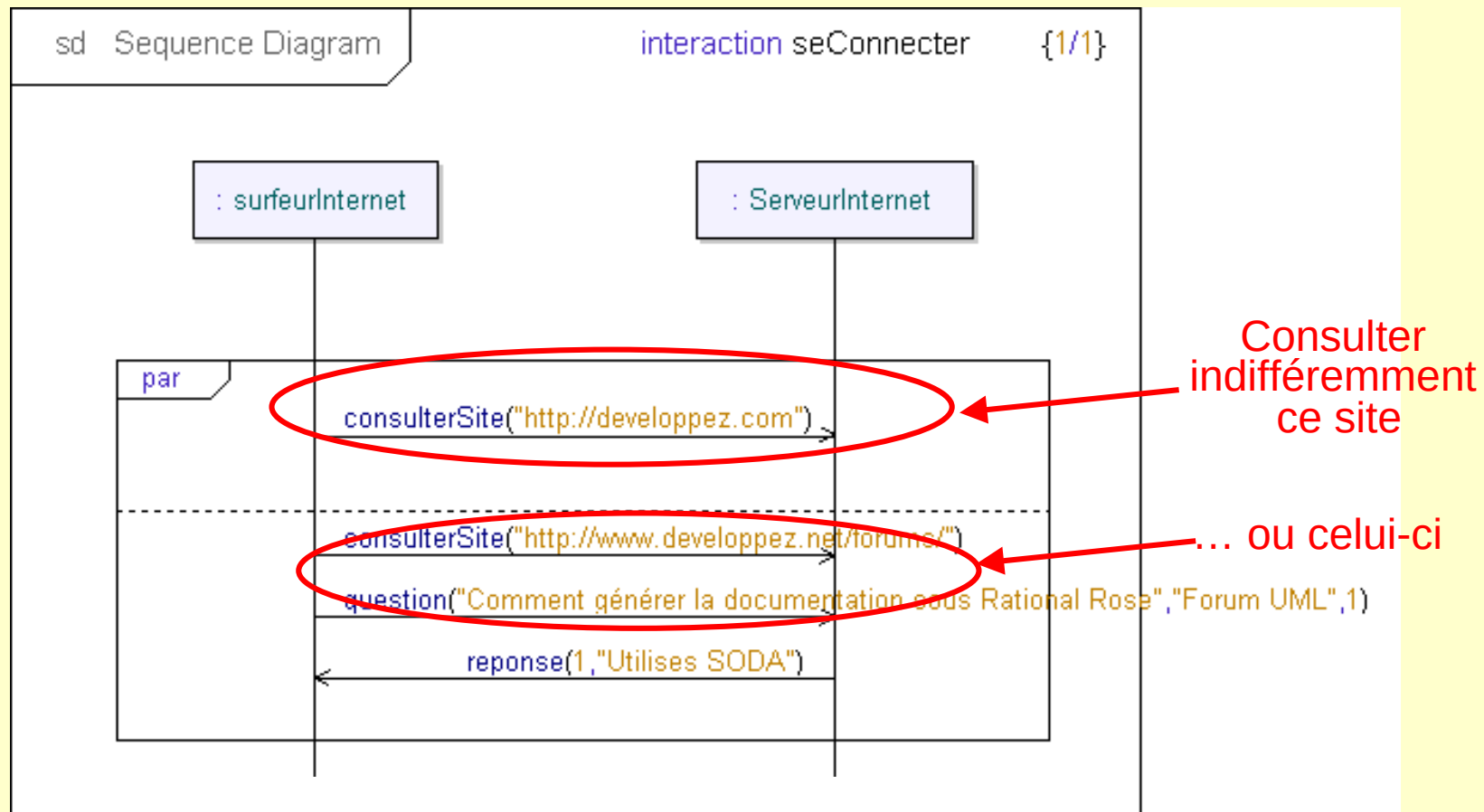
Fragments combinés *break*

- un comportement d'exception
- *L'utilisateur peut choisir de consulter l'aide au lieu de saisir son code. Le flot d'interactions standard s'interrompt pour enclencher sur cette interaction.*
- Autrement, 2 différents diagrammes de séquence



Fragments combinés *par*

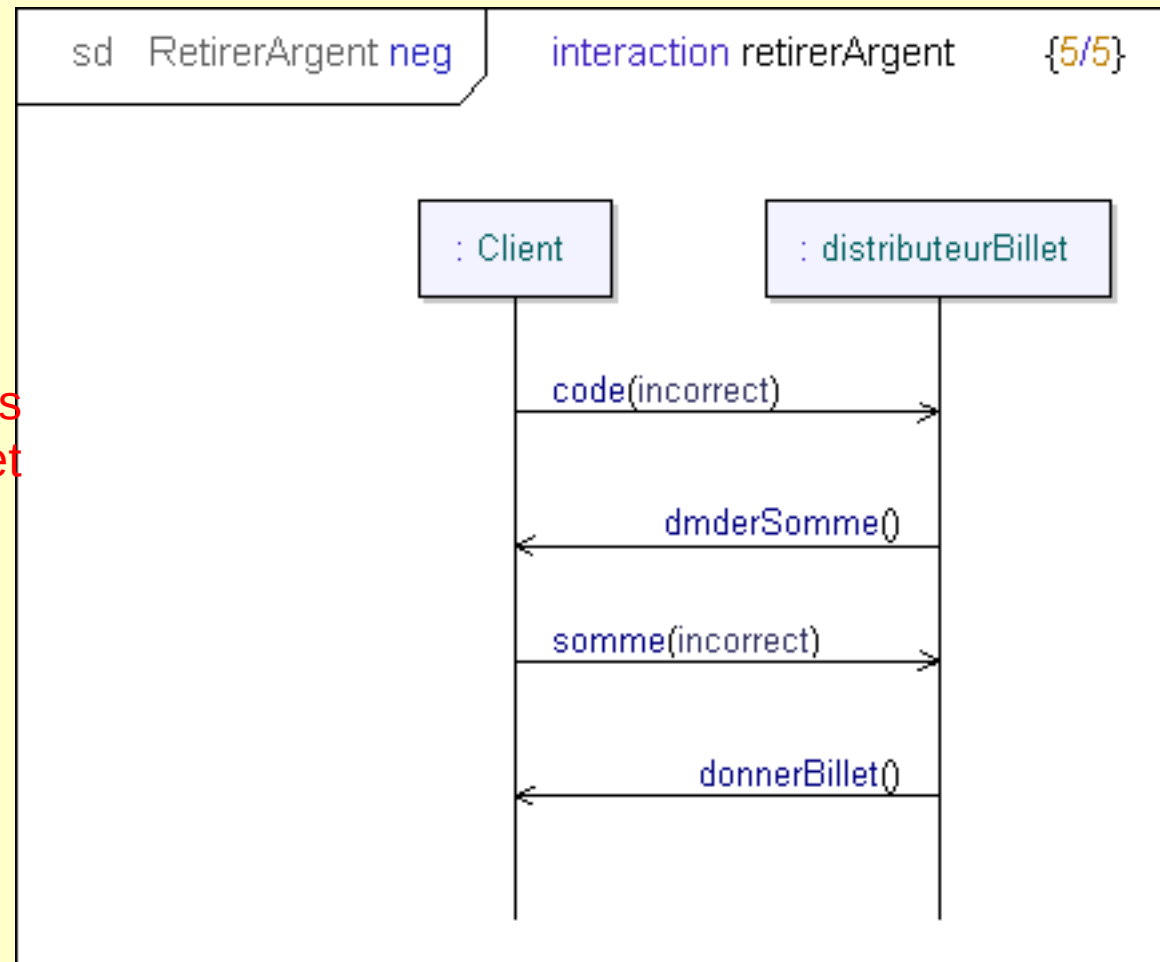
- Interactions parallèles



Fragments combinés *neg*

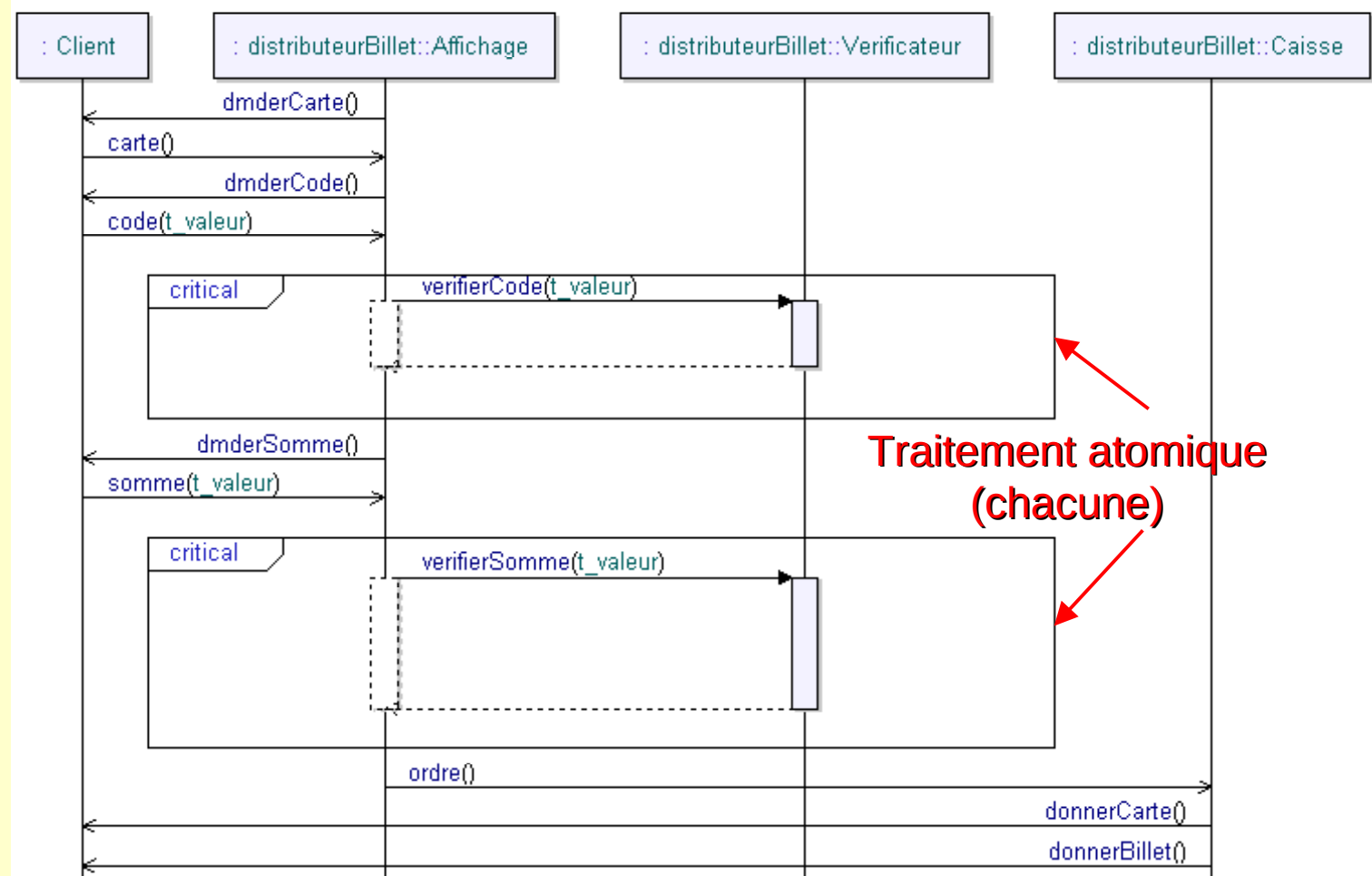
- Interactions invalides

On ne souhaite pas que l'utilisateur puisse obtenir des billets avec un code erroné et une somme demandée incorrecte.



Fragments combinés *critical*

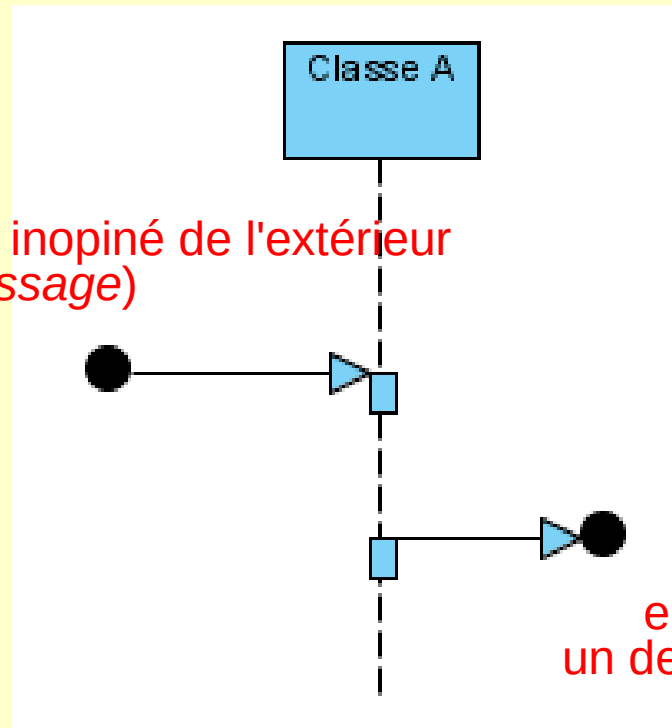
Interactions ne devant pas être interrompues par d'autres interactions du même diagramme.



Traitement atomique
(chacune)

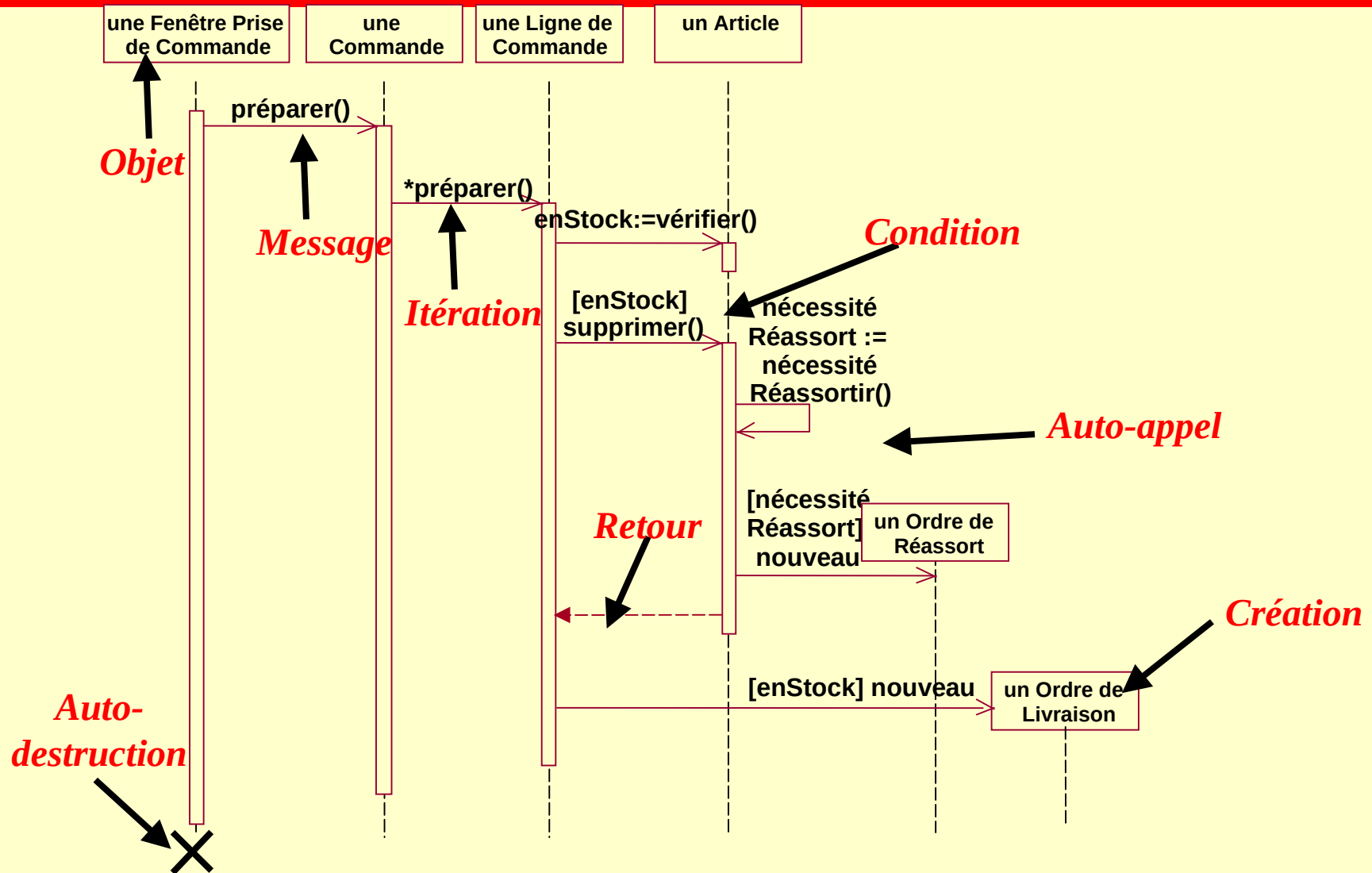
Messages

déclenchement message inopiné de l'extérieur
(*found message*)

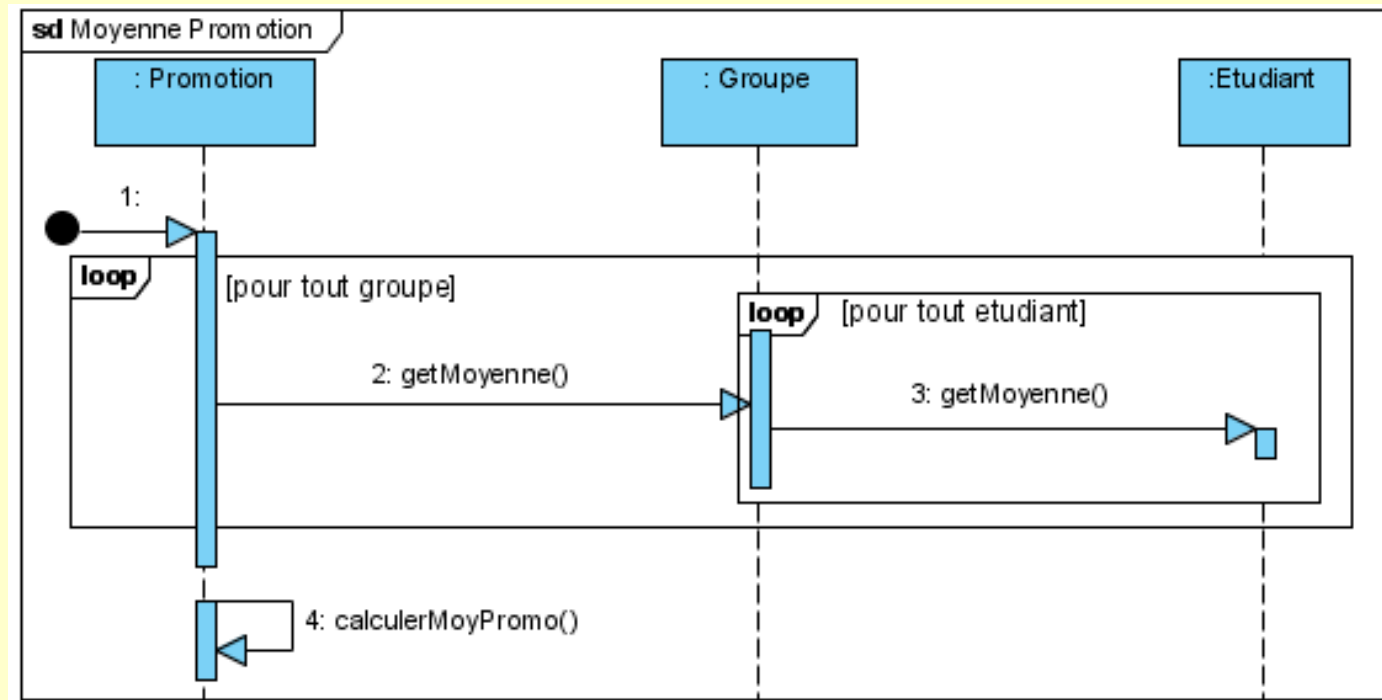


envoi message vers
un destinataire non contacté
(*lost message*)

Diagramme de séquence en UML 1.4



Petit exercice



Donner le code Java associé ...

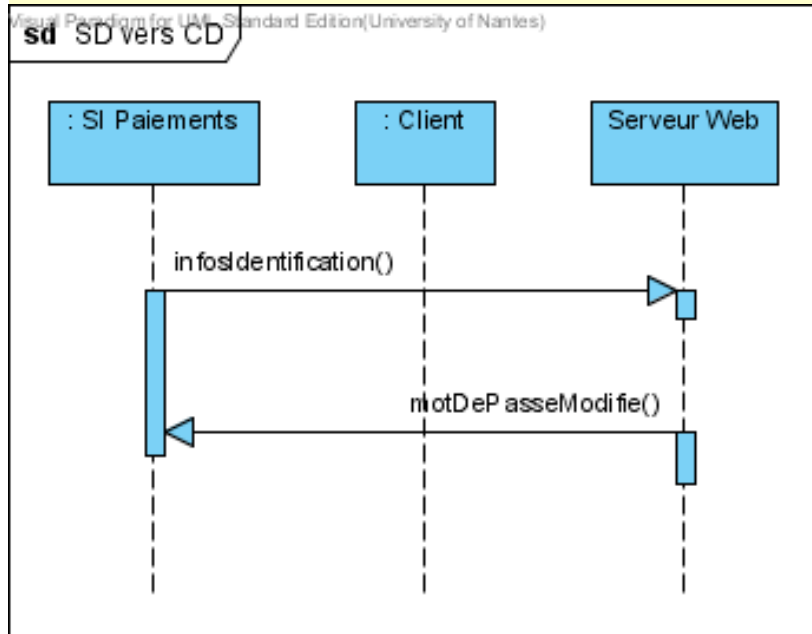
Utilité

- Une bonne conception = petites méthodes réparties dans différentes classes.
- Représentation des envois de messages est le flux de contrôle global. Sa représentation est difficile.
- Validation des classes et des cas d'utilisation : toute opération du diagramme de séquence doit être définie au niveau des diagrammes de classes.
- Représentation de processus concurrents.

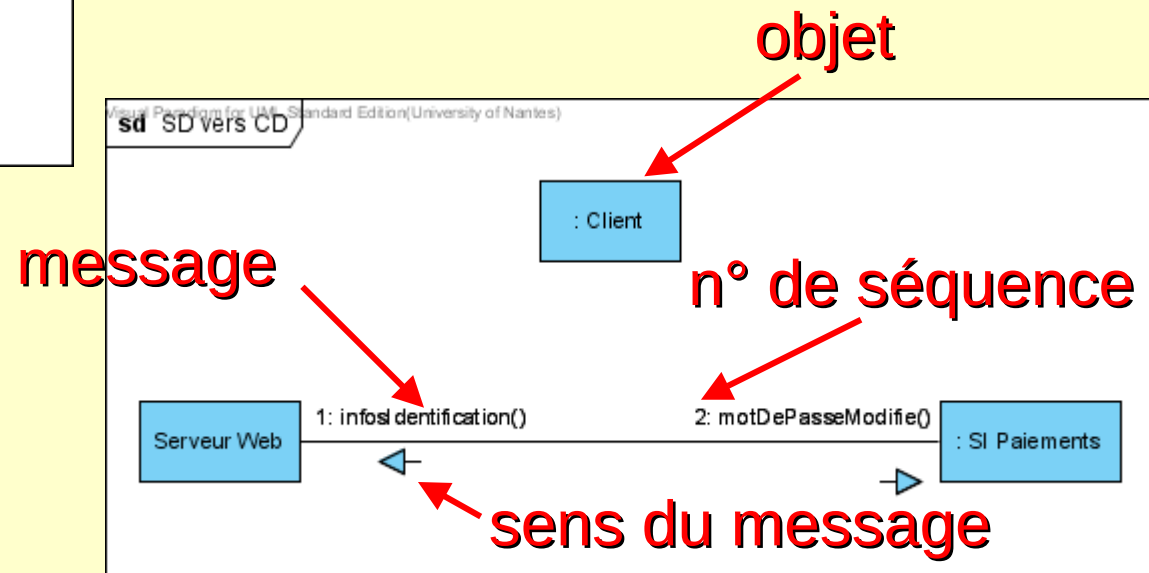
Diagramme de communication

- ... ou de Collaboration
- Seconde forme de diagramme d'interactions.
- Mêmes informations qu'un diagramme de séquences mais :
 - Ne privilégie pas l'aspect temporel
 - Privilégie l'aspect spatial
- Met en évidence les liens statiques entre objets.
- Ordre (séquence) représentée par la numérotation des envois de messages (déclenchement d'opérations).

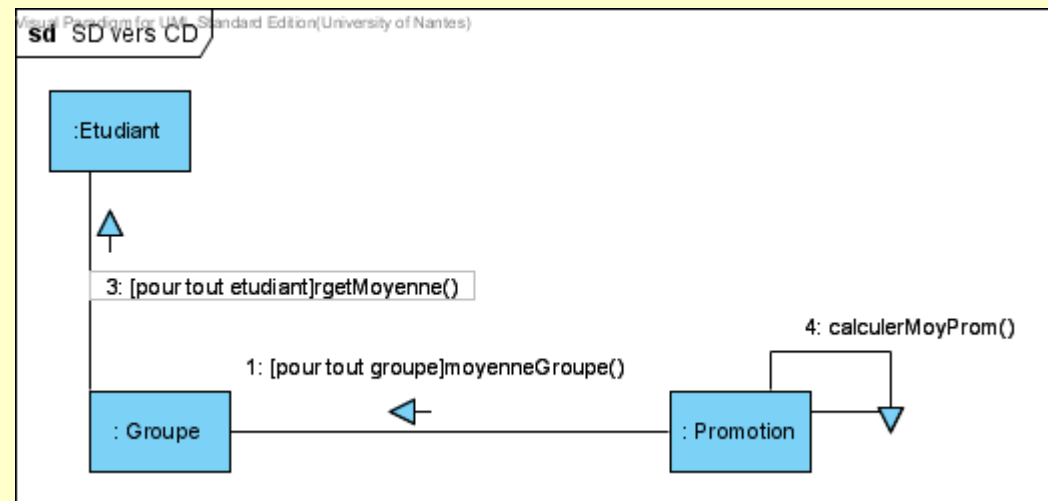
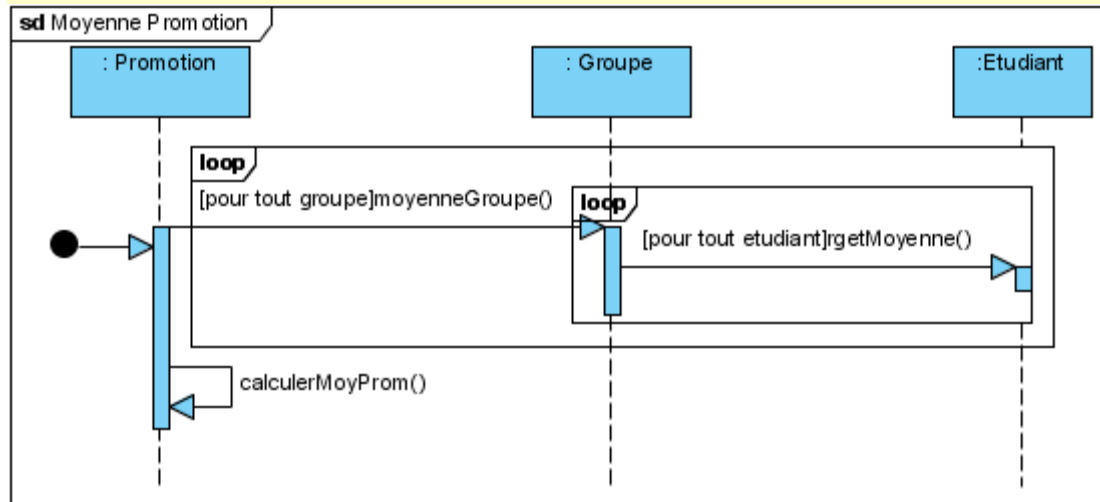
DS versus DC



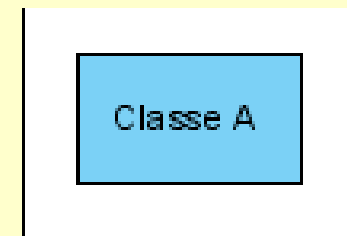
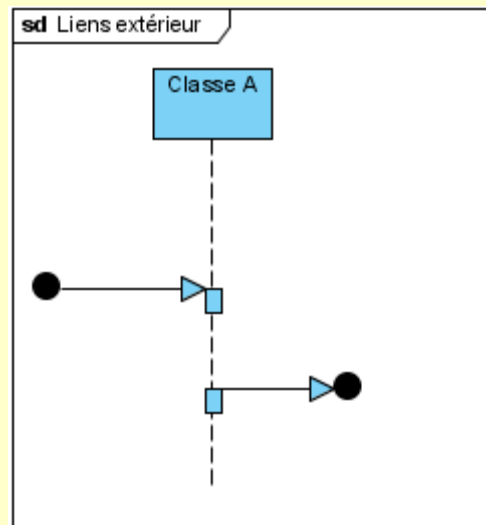
Généralement, se déduit
du diagramme de
séquences (sur un atelier)



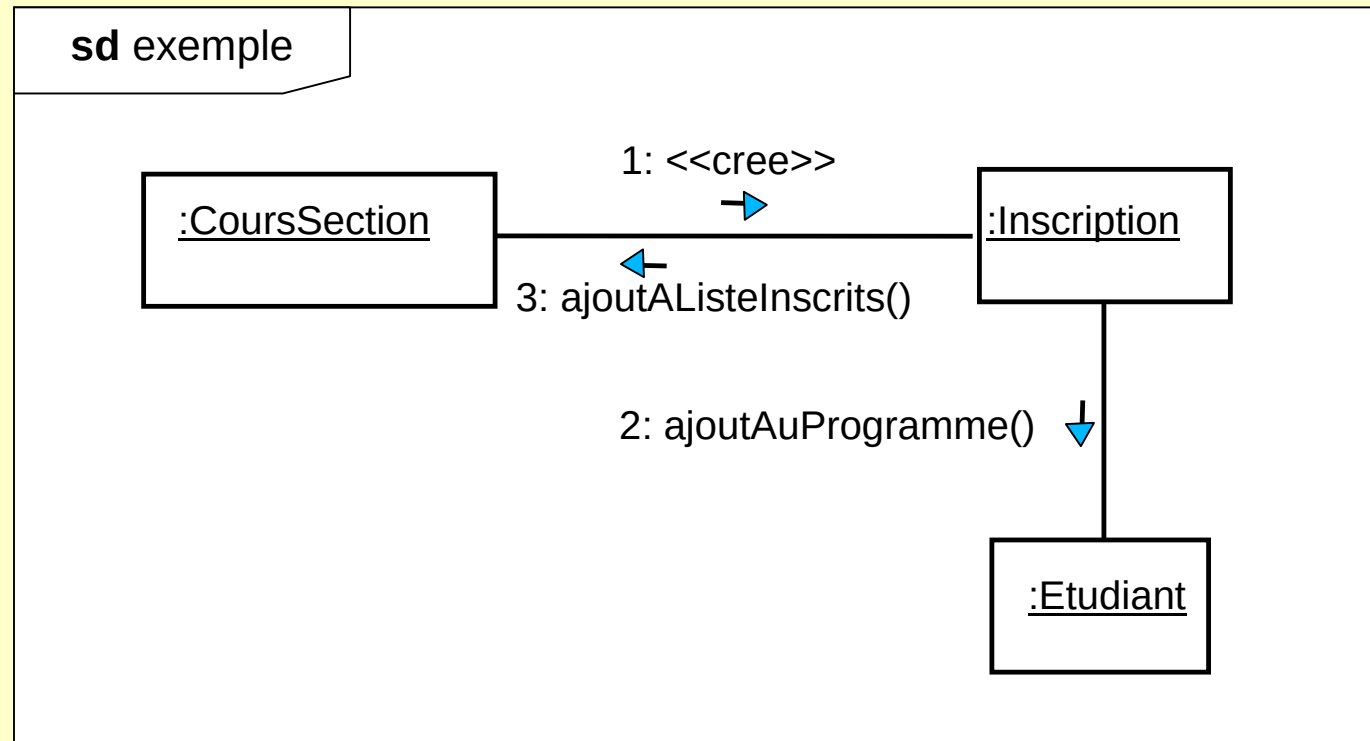
DS versus DC



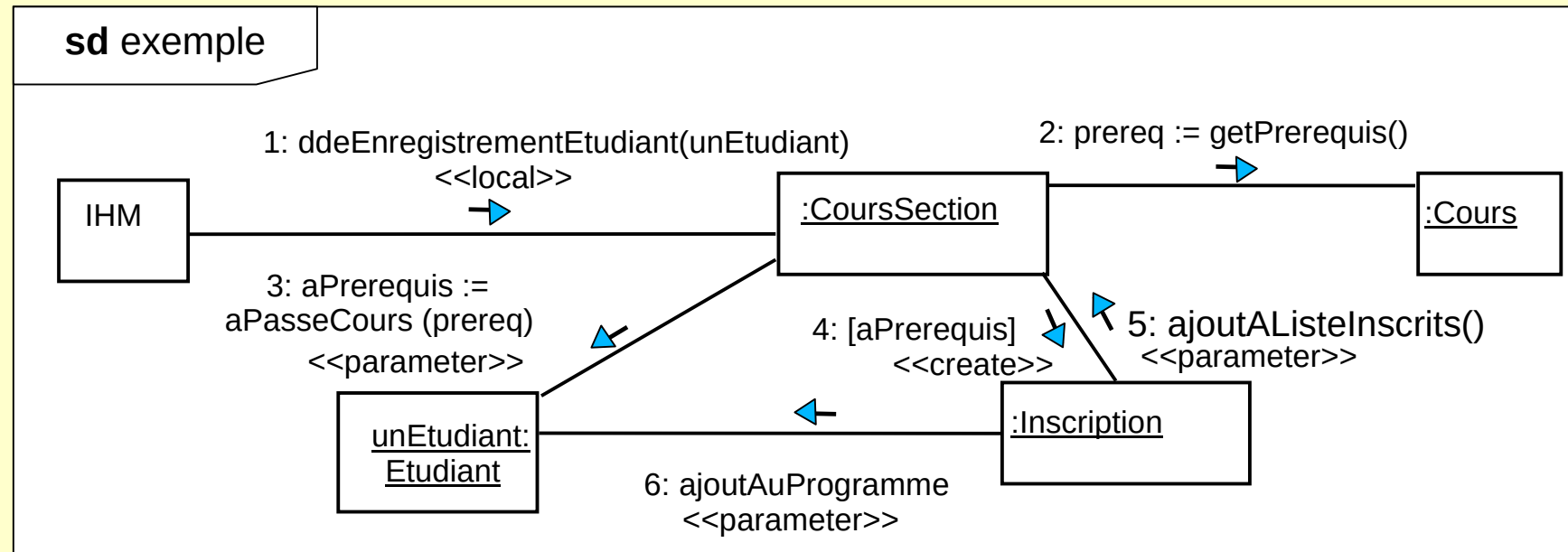
DS versus DC



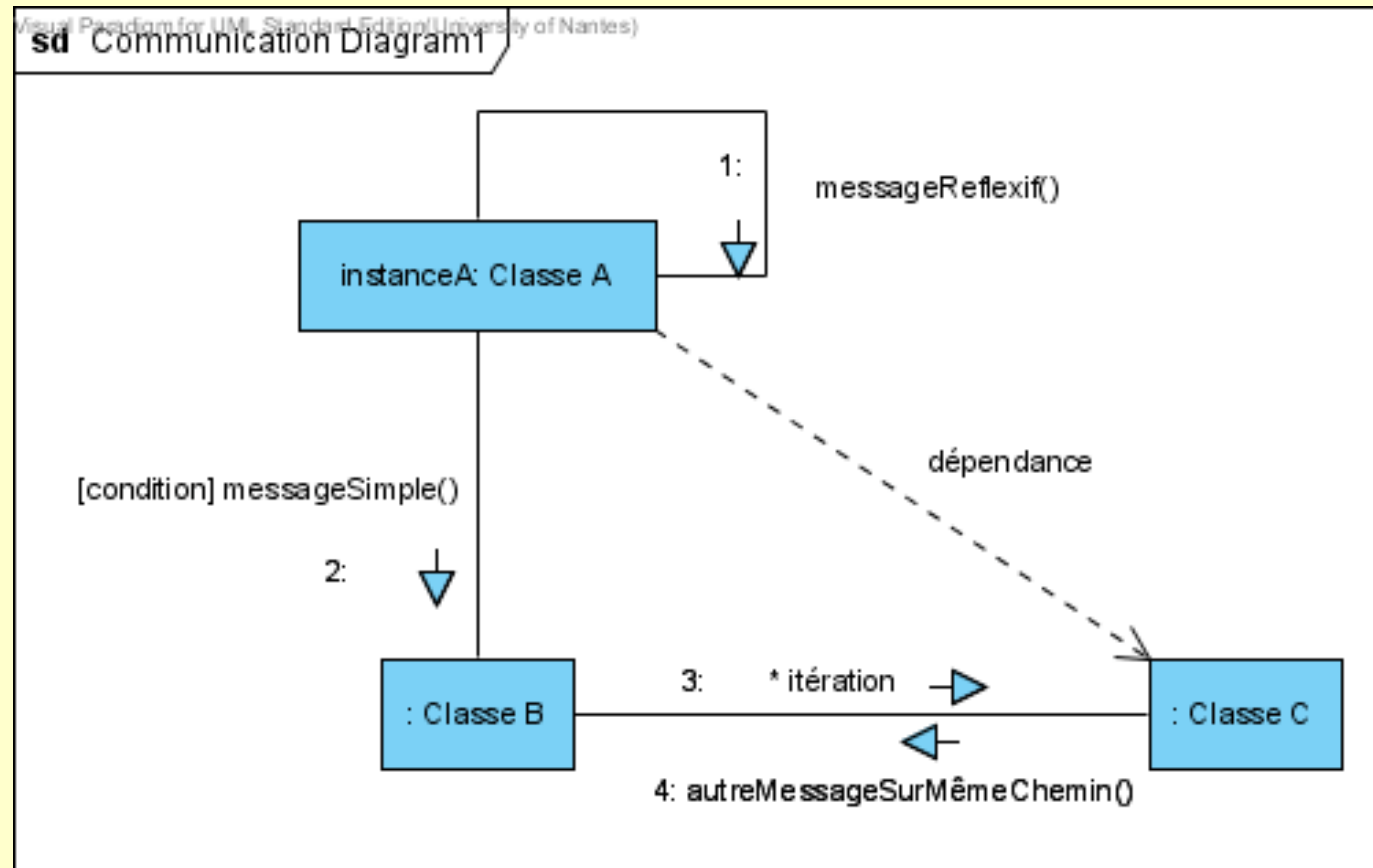
Exemple



Exemple plus détaillé



Suite formalisme



Communication au sein de diagrammes d'interactions

- Un 'chemin' de communication doit exister entre deux objets pour qu'ils interagissent par envois de messages
- Différents chemins :
 - Association entre classes
 - Attention à la navigabilité
 - Sur une association, si tous les messages sont envoyés dans la même direction, l'association peut alors être rendue unidirectionnelle => *impact sur le diagramme de classes*

Communication au sein de diagrammes d'interactions

- Différents 'chemins' (suite) :
 - Le récepteur est dans une variable locale de la méthode émettrice
 - Objet créé par la méthode émettrice
 - Un résultat précédent a retourné un objet .
 - Le stéréotype à utiliser est «local» ou [L].
 - Une référence à l'objet récepteur a été reçu comme paramètre par la méthode émettrice
 - S'appuie sur une dépendance dans le diagramme de classes
 - Le stéréotype à utiliser est «parameter» or [P].

Diagrammes séquence - Collaboration

- Lequel choisir ?
 - Choix du concepteur.
 - Ateliers génèrent les diagrammes de collaborations à partir des diagrammes de séquence.
- Quand les utiliser ?
 - Visualiser le comportement de plusieurs objets d'un seul cas d'utilisation.
 - Mettent en évidence les collaborations entre objets.
- Limites :
 - Pas clair pour observer la définition précise du comportement d'un objet.
 - Comportement d'un objet dans plusieurs cas d'utilisation, utilisez un diagramme d'états-transitions.
 - Un comportement réparti, choisissez un diagramme d'activité.

Diagrammes d'états-transitions

State-chart diagrams

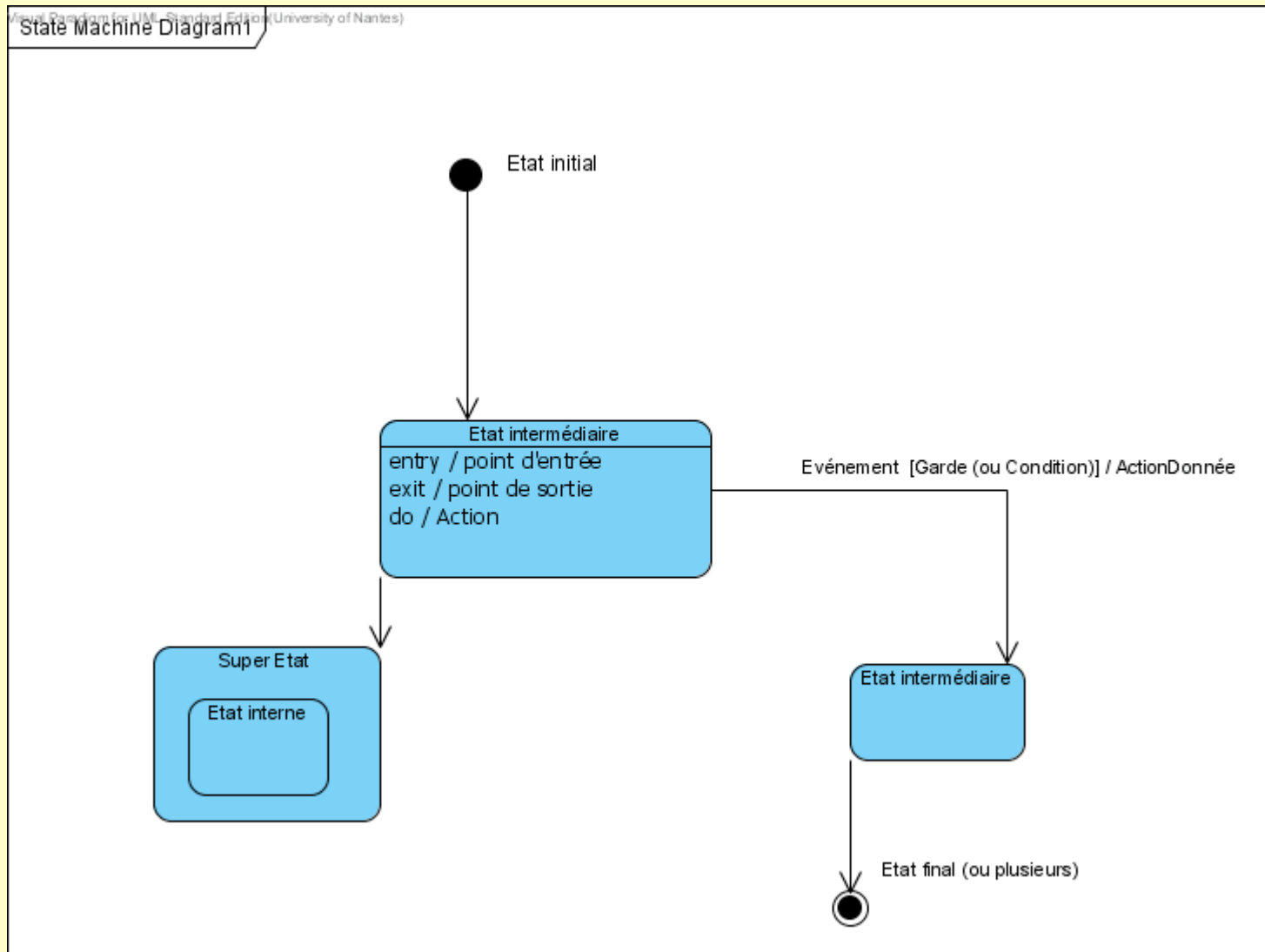
Présentation

- Automates à états finis
- Technique familière de description du comportement d'un système. En C/POO, généralement :
 - Décrit le comportement et tous les états possibles des instances, d'une classe, tout au long de leurs vies
 - Les transitions d'un état à un autre
 - Diagramme rattaché à cette classe.
- Particulièrement utile dans la modélisation des systèmes réactifs
- Syntaxe : *Evénement*[*Garde*]/*Action*.

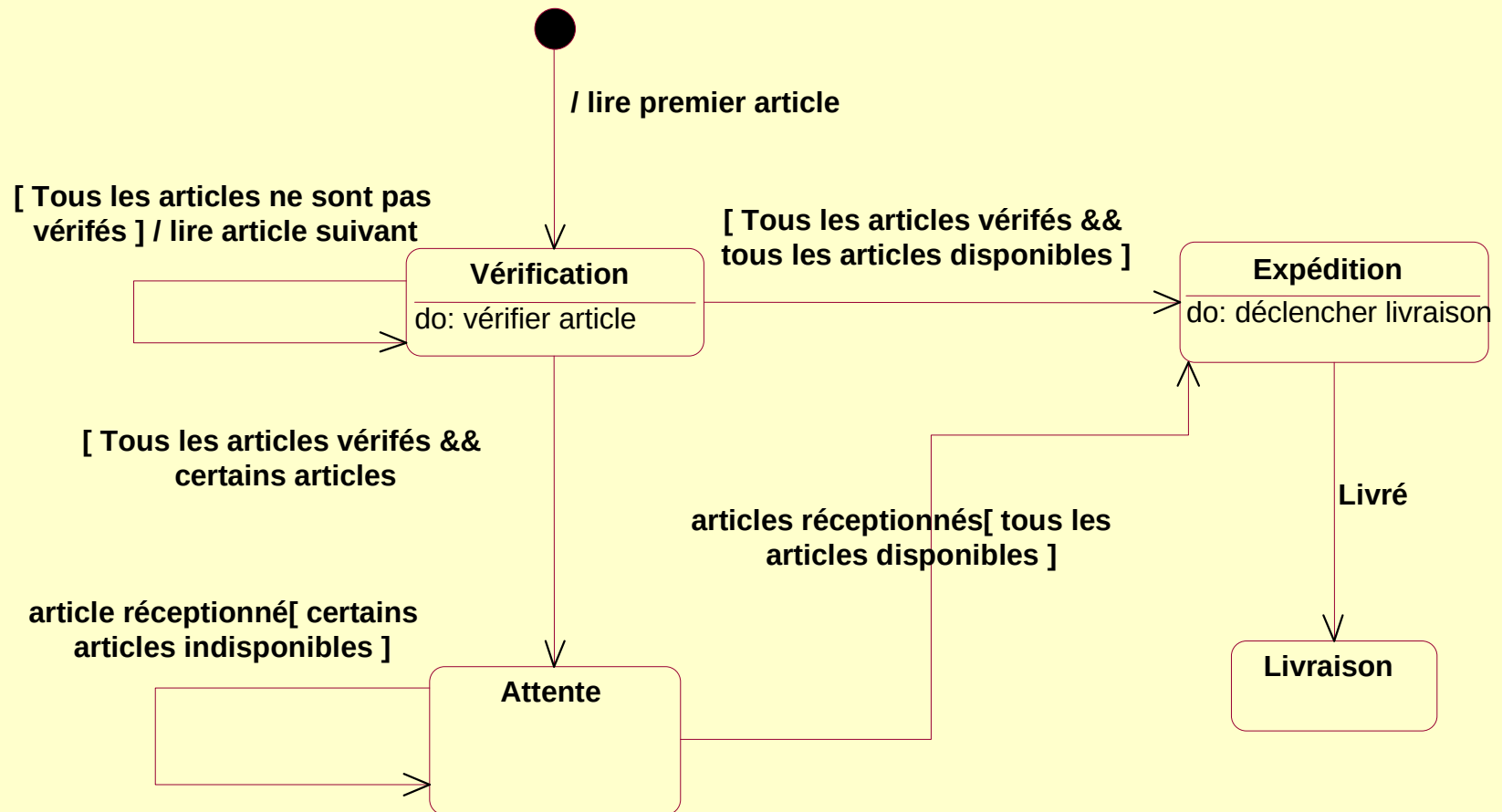
Présentation

- Nécessaire pour documenter les classes qui ont :
 - des états clairement identifiables
 - un comportement complexe
 - beaucoup d'événements (signaux)

Formalisme graphique

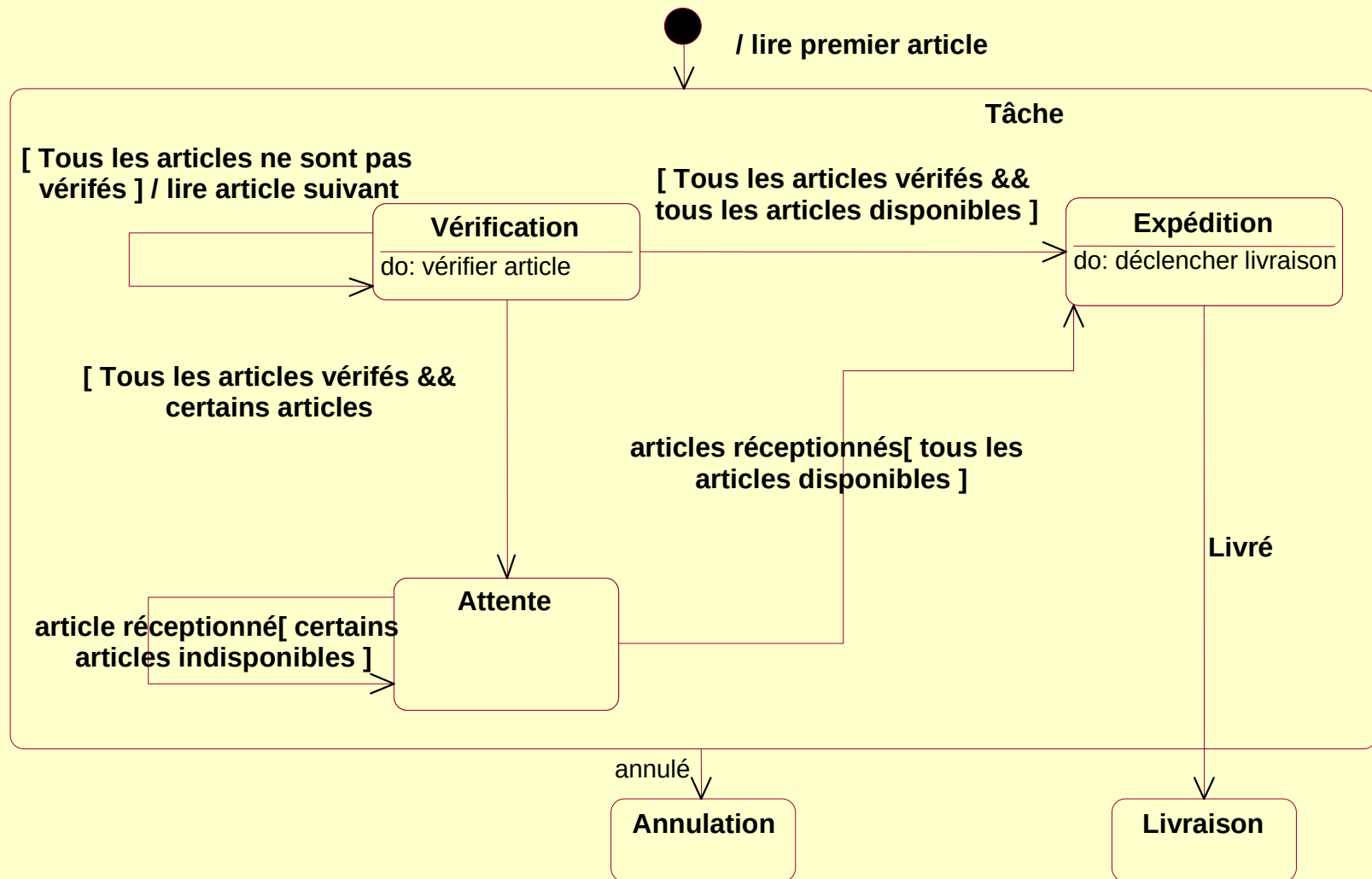


Un exemple



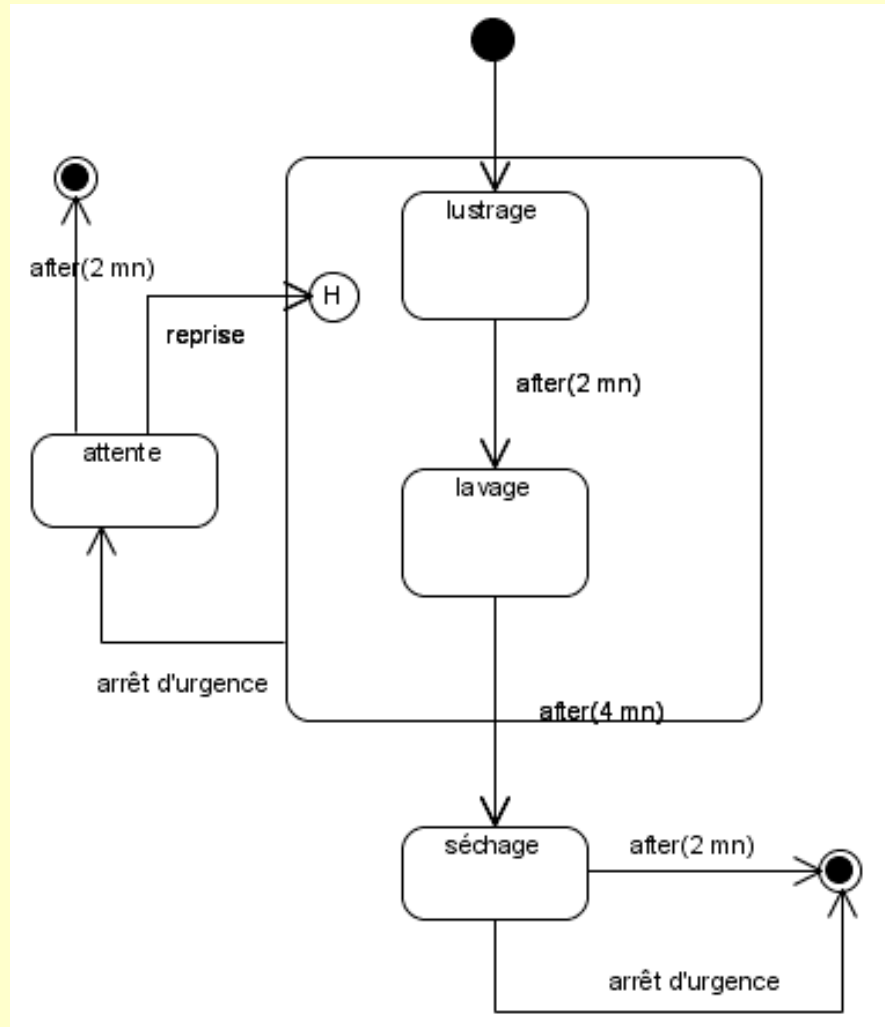
Les états d'un objet de la classe Commande

Avec super-état



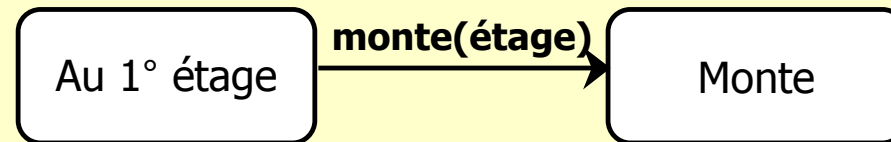
Super-état

- Un **super-état** :
 - est un état qui **contient d'autres états**
 - a un **historique**, *càd* une mémoire du dernier sous-état actif.

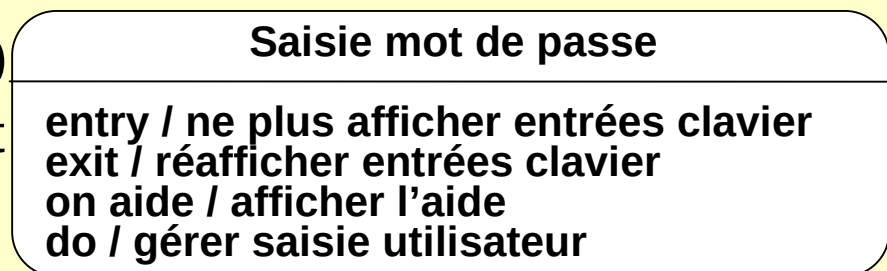


Événement/action

- Pour lier un événement à une action
 - syntaxe : événement/action.
 - Une action se réalise de façon instantanée
- call action : lorsqu'une transition se produit

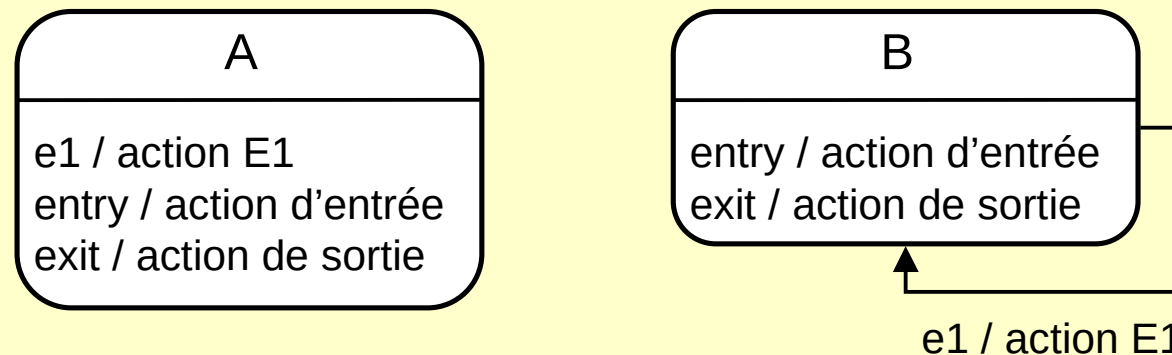


- actions internes : des méthodes prédéfinies et de l'utilisateur permettent de le faire
 - **entry** (exécution à l'entrée)
 - **do**(action récurrente dans l'état)
 - **on <événement>**(si événement valide alors exécution)
 - **exit**(exécution à la sortie)



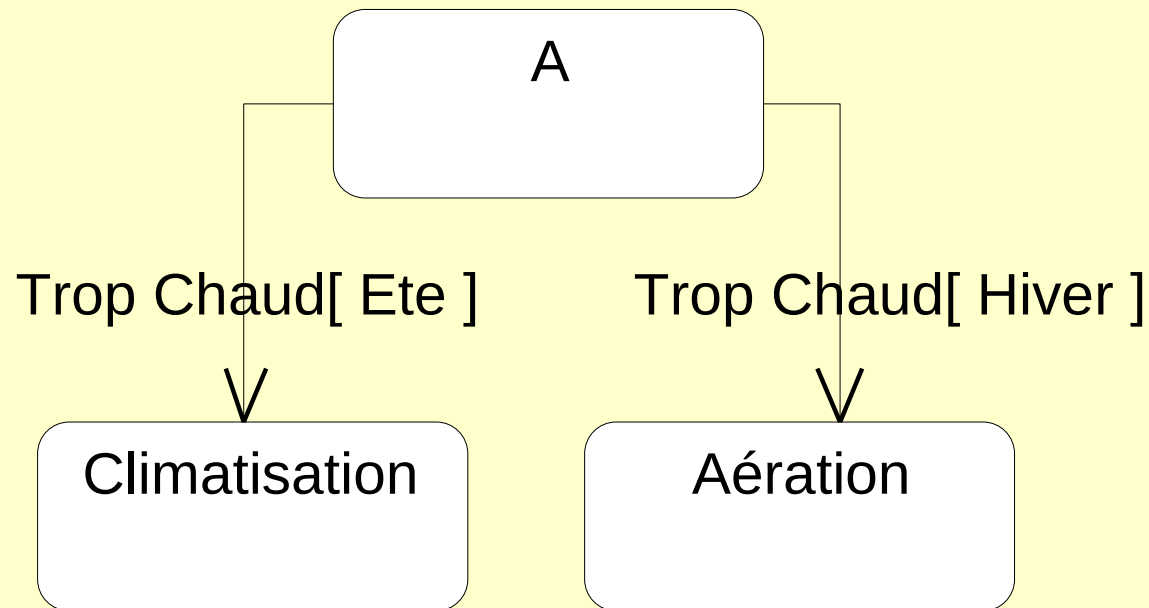
Actions internes

- Différence fondamentale entre action interne et transition réflexive
 - l'action interne n'entraîne pas l'exécution des actions d'entrée et de sortie



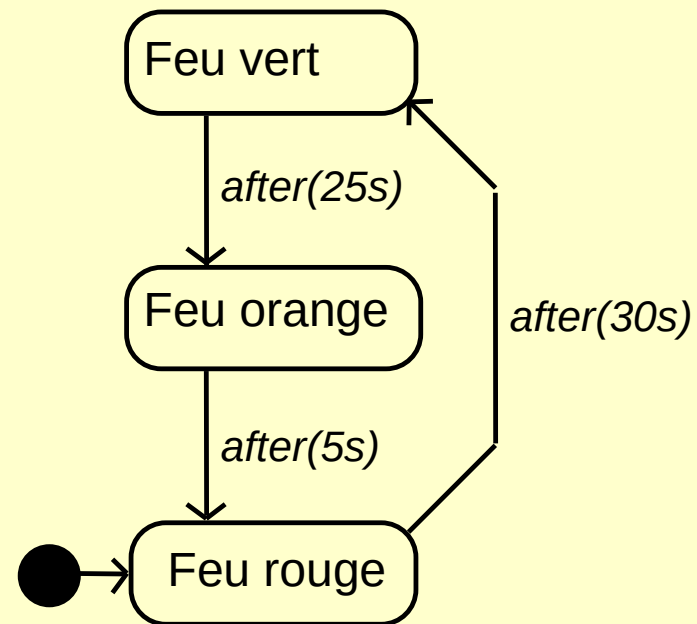
Concept *garde*

- une garde (*guard*) est une condition qui doit être satisfaite pour que la transition ait lieu



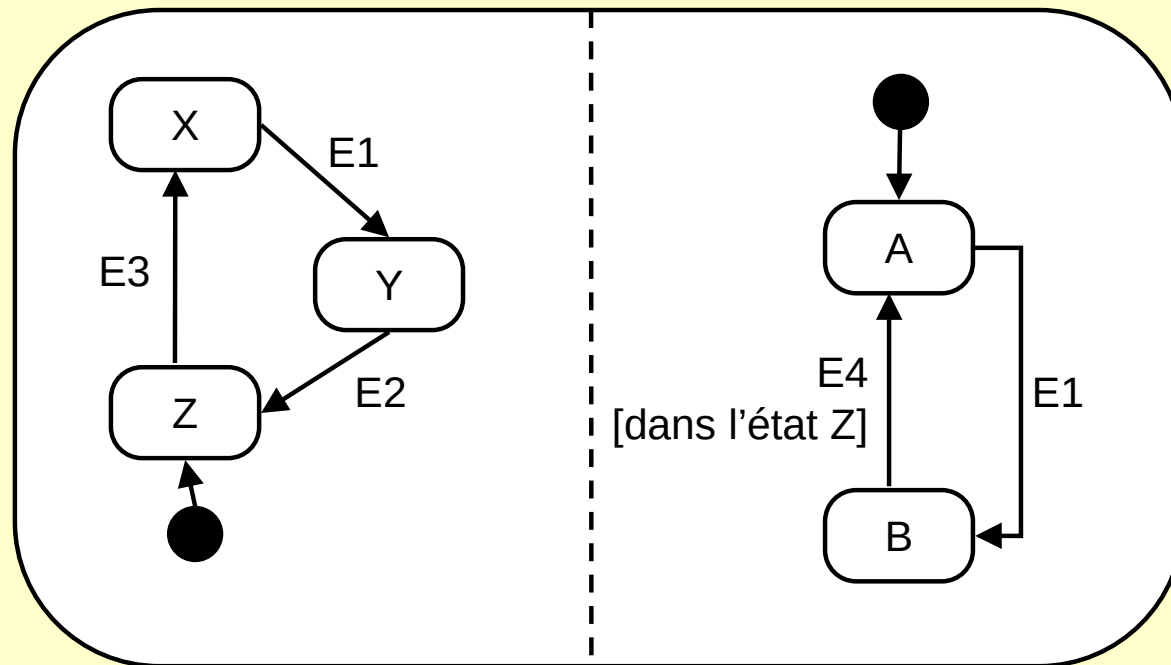
Exemple

Feux de signalisation simplifié



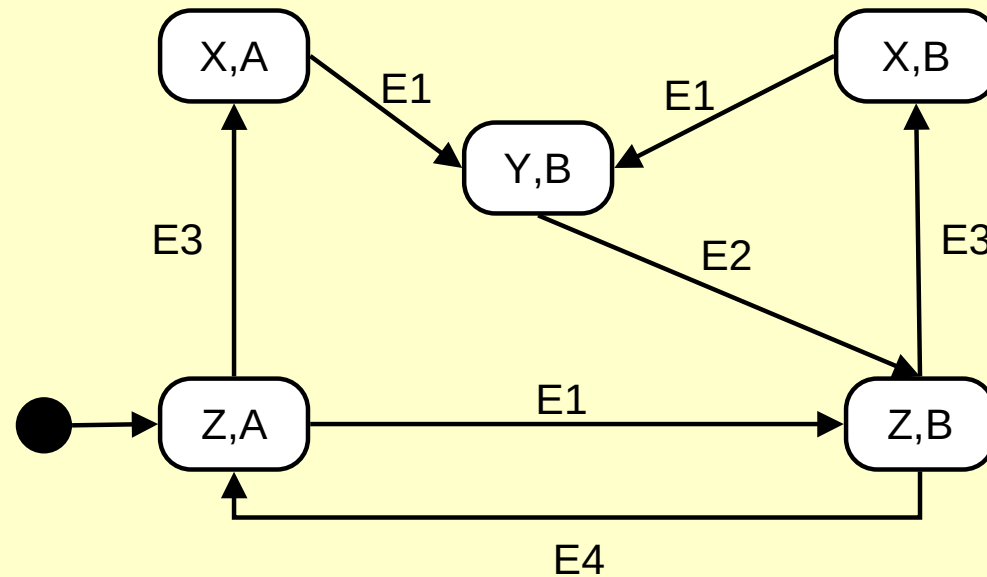
États concurrents

- On objet peut être simultanément dans différents sous-états
 - représente une forme de parallélisme entre automates
- Exemple



États concurrents

- Equivalent à :



Utilisation

- Cycle de vie des objets d'une même classe.
- Donc son comportement au travers des différents cas d'utilisation.
- Ne conviennent pas pour la description de collaborations entre objets.
- Utilité de combiner avec d'autres techniques: les diagrammes d'interactions.
- Ne les utiliser que pour les cycles de vie complexes.

Autres diagrammes d'UML

Conception Orientée Objet – UML

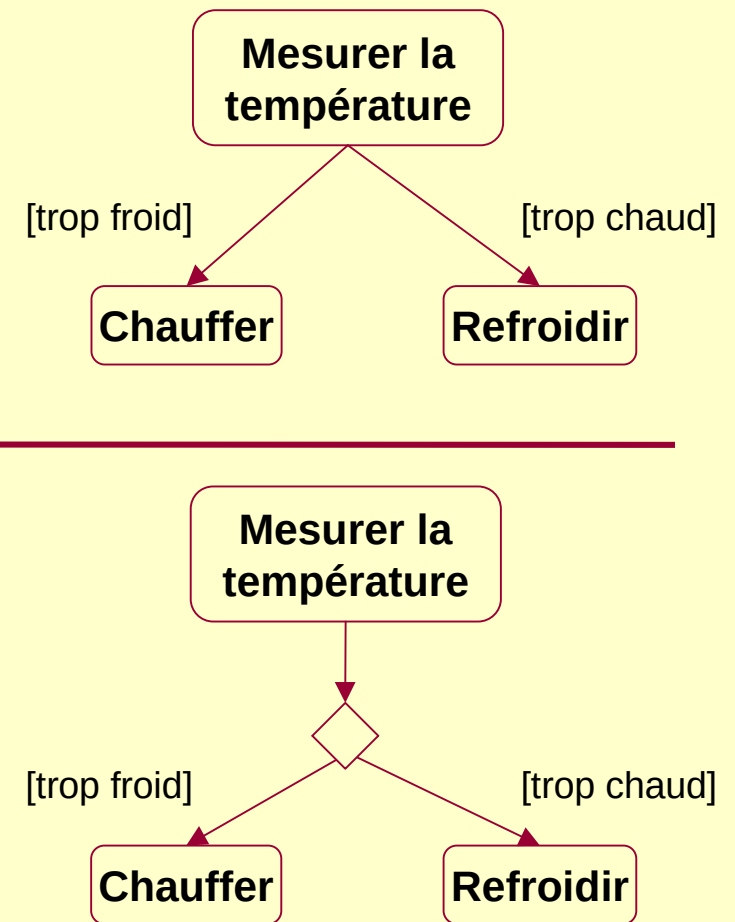
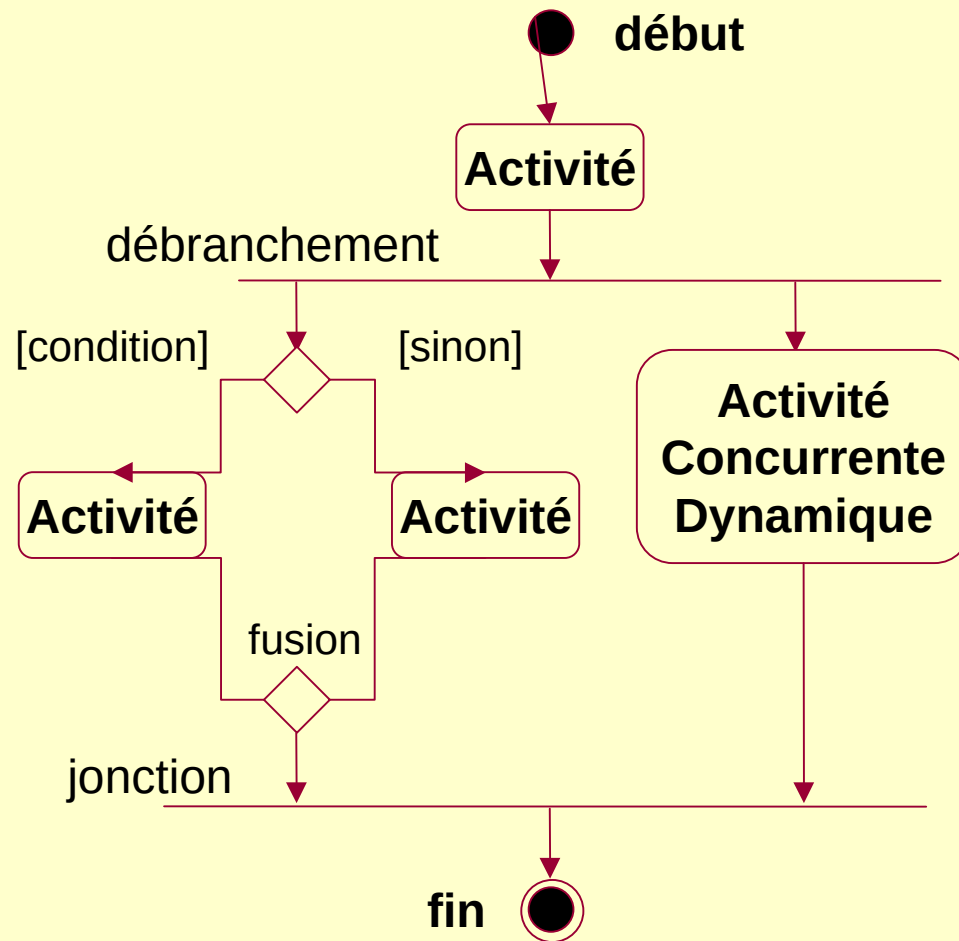
Autres diagrammes

- Vue dynamique :
 - Diagrammes d'activité
- Vue physique:
 - Diagrammes de composants
 - Diagramme de déploiement

Diagrammes d'activité

- Variante des diagrammes d'états-transitions. Le diagramme d'activité décrit le déroulement du système.
- Organisé par rapport aux actions
- Destiné à représenter le comportement interne d'une méthode ou d'un cas d'utilisation
- Formalisme graphique: identique à celui du diagramme d'états-transitions:
 - Une activité comme un état
 - Chaque activité est une étape particulière d'exécution
 - Les transitions automatiques comme des flèches

Formalisme et exemple



Diagrammes physiques

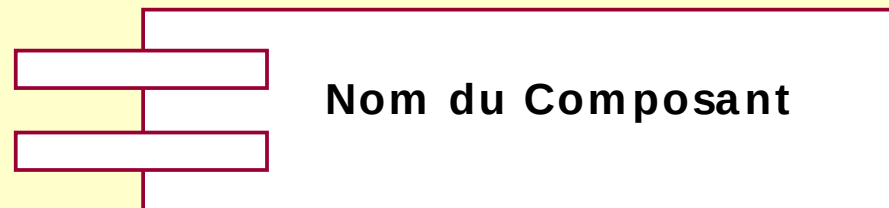
- UML propose deux types de vue pour représenter des unités d'implémentation
 - Le diagramme de composants : décrit les composants logiciels, voire physiques, constituant le système.
 - Le diagramme de déploiement : décrit l'architecture physique et le déploiement physique du système développé

Les composants

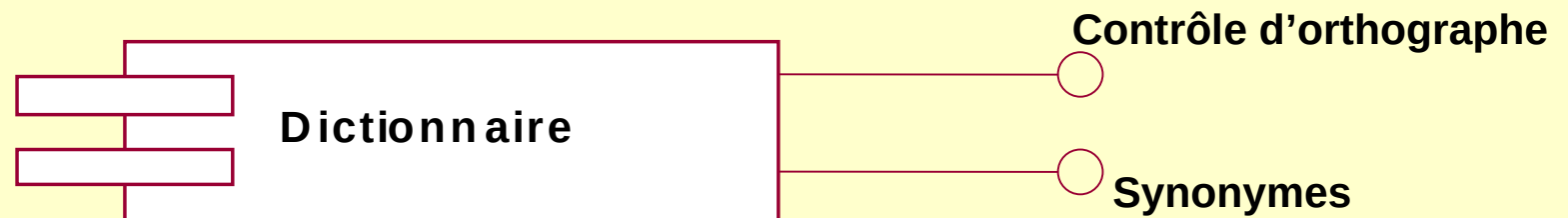
- Un composant: unité physique d'implémentation avec des interfaces (liste d'opérations) bien définies
- Chaque composant contient l'implémentation de certaines classes du système
- Les composants bien formés ne dépendent pas directement d'autres composants mais des interfaces qu'ils supportent

Composants: formalisme

- Composant simple:



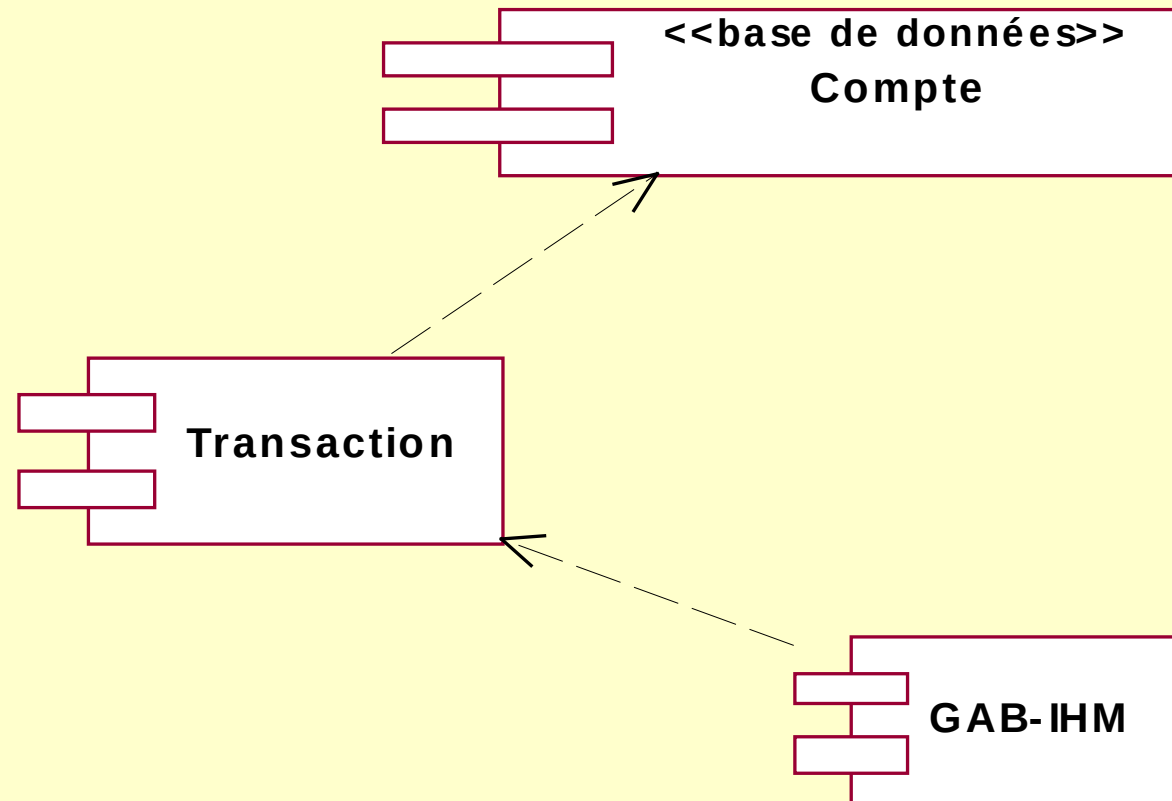
- Composant avec interfaces:



Digramme de composants

- Montre l'implémentation des éléments du modèle de conception (classes) par les composants, les interfaces et les dépendances entre composants.
- Le diagramme de composants peut avoir deux formes. Il peut montrer :
 - Un ensemble de composants disponibles (bibliothèques de composants) avec leurs dépendances
 - Le système configuré avec les composants (pris dans la bibliothèque) utilisés pour le construire

Exemple

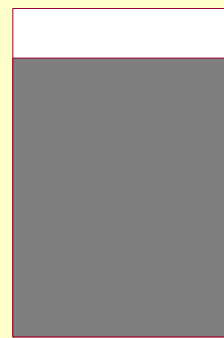


Autres symboles

**Spécifications de
sous-programme**



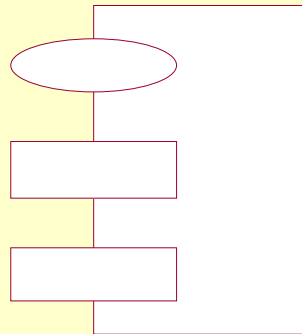
**Corps du
sous-programme**



Programme



**Spécifications de
paquetages**



**Corps de
paquetages**

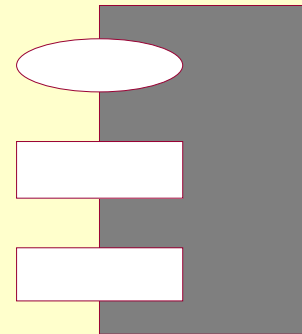


Diagramme de déploiement

- Montre l'arrangement physique des ressources d'exécution: les machines et leurs interconnexions
- Les ressources d'exécution sont dits des nœuds
- A l'exécution, les nœuds peuvent contenir des composants et des classes
- L'attribution des composants aux nœuds:
 - Peut être statique
 - Peut migrer parmi les nœuds

Exemple

